

CIA - Key Objective / Requirements

⚡ Sicurezza Informatica - NIST

L'insieme delle misure che garantiscono **confidenzialità, integrità e disponibilità** di un sistema informatico nella sua interezza. Questo include hardware, software, firmware e tutte le informazioni che il sistema memorizza, elabora e trasmette.

Queste tre proprietà fondamentali costituiscono la cosiddetta **Triade CIA** (*Confidentiality, Integrity, Availability*) e significano:

- **Confidenzialità (Riservatezza):** Assicura che le informazioni private o sensibili non vengano divulgate a persone o entità non autorizzate. Include la tutela della **privacy**, garantendo che l'utente abbia il pieno controllo su quali dei suoi dati personali vengano memorizzati nel sistema e a chi vengano inviati.
- **Integrità:** Garantisce che i **dati** e il **sistema** stesso non subiscano alterazioni. Le informazioni devono poter essere modificate esclusivamente in modo autorizzato, preservandone l'accuratezza.
- **Disponibilità:** Assicura un accesso affidabile e tempestivo alle informazioni e ai servizi per gli utenti autorizzati, garantendo che non vi siano interruzioni del servizio.

A questa triade, negli ultimi anni, si è aggiunto:

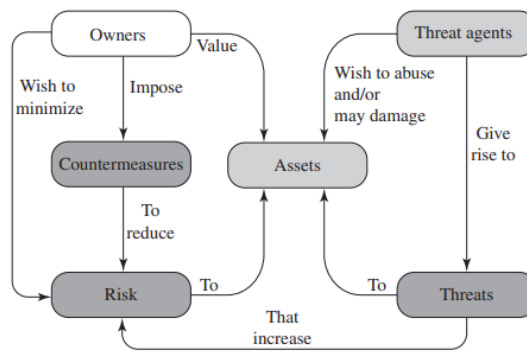
- **Autenticità:** garantisce che la sorgente sia fidata e che durante la trasmissione vi sia stata confidenzialità.
- **Accountability:** assicura la possibilità di tracciare e risalire a quale utente corrisponde un'azione data.

Livelli d'impatto

Se la CIA viene violata, l'impatto può essere:

- **Basso** (limitato): se le perdite causano danni lievi.
- **Moderato:** se le perdite causano danni seri, compromettendo l'operatività.
- **Alto:** se le perdite causano danni catastrofici, bloccando l'operatività.

Un modello per la computer security



Nel diagramma sono indicati:

- **Asset** (Risorsa): È quello che ha un valore nel sistema (Hardware, software, Dati ecc...)
 - **Vulnerabilità** (di un asset): Una debolezza nel sistema che può essere sfruttata. Ne esistono di diversi tipi:
 - **Corrupted**: Dati alterati
 - **Leaky**: Dati trapelati
 - **Unavailable**: Sistema non disponibile
 - **Threat**: Una qualsiasi «situazione» che potrebbe causare danni di qualsiasi tipo.
 - **Attack**: Quando un Threat Agent ovvero un avversario sfrutta attivamente una vulnerabilità.
 - **Countermeasures**: Tecniche o dispositivi utili a prevenire gli attacchi o a ridurne gli effetti.
- Possiamo anche **calcolare il rischio** grazie alla formula:

$$R = P \times D$$

Dove:

- **P** Probabilità che l'evento si verifichi
- **D**: Impatto dell'evento

Classificazione degli Attacchi

La prima distinzione che si può fare è quella della provenienza dell'attacco, se dall'interno (**insider**) o dall'esterno (**outsider**).

Gli attacchi si dividono in:

- **Passivi**: L'attaccante osserva e raccoglie informazioni senza alterare il sistema, questi sono molto difficili da rilevare e l'unica difesa è la prevenzione. Ne esistono di due tipi:
 - **Release of Message Contents**: Leggere il contenuto di un messaggio.
 - **Traffic Analysis**: Anche se il messaggio è cifrato, l'attaccante osserva chi sta parlando con chi e guardando anche la dimensione del messaggio, la frequenza e altri dati può ricavare qualche informazione.
- **Attivi**: L'attaccante altera i dati o il funzionamento del sistema:
 - **Replay**: Catturare un dato e rinviarlo successivamente per ottenere accesso.
 - **Masquerade**: Fingersi qualcun altro.

- *Modification of Messages*: Alterare parti legittime di un messaggio o ritardarlo.
- *Denial of Service (Dos)*: Interrompere il servizio di un sistema sovraccaricandolo di richieste.

Vediamo adesso le *conseguenze e i tipi di attacco che le producono* (secondo lo standard RFC4949):

- **Unauthorized Disclosure** - Vengono divulgati dati da persone non autorizzate, attacchi possibili:
 - *Exposure*: i dati vengono divulgati direttamente ad un'autorità non autorizzata
 - *Interception*: un entità non autorizzata accede ai dati sensibili che viaggiano tra autorità autorizzate.
 - *Inference*: deduzione di dati da altri segnali
 - *Intrusion*: un entità non autorizzata elude le protezioni di sicurezza
- **Deception**:
 - *Masquerade*: fingersi un'altra persona
 - *Falsification*: alterare i dati per ingannare
 - *Repudiation*: negare di aver compiuto un'azione
- **Disruption** - Interruzione delle funzionalità del sistema:
 - *Incapacitation*: disabilitare un componente
 - *Corruption*: alterare i dati che permettono il funzionamento del sistema
 - *Obstruction*: ostacolare i servizi
- **Usurpation** - Viene preso il controllo del sistema da parte di utenti non autorizzati:
 - *Misappropriation*: prendere il controllo logico o fisico di una risorsa
 - *Misuse*: utilizzare un componente per svolgere operazioni dannose.

Requisiti di Sicurezza

Le contromisure alle vulnerabilità e minacce possono essere classificate e categorizzate in differenti modi, possiamo classificarle in funzione dei requisiti funzionali, il FIPS 200 le divide in:

- **Requisiti di Sicurezza Tecnici**: Ad esempio controllo degli accessi, autenticazione e integrità del sistema.
- **Funzionali**: Formazione del personale, valutazione dei rischi, sicurezza fisica.
- **Misti**: Gestione della configurazione, risposta agli incidenti e protezione dei media.

Principi fondamentali di Design per la sicurezza

Questi principi sono fondamentali per chi progetta architetture di rete o sviluppa software:

- *Economy of Mechanism*: I sistemi di sicurezza devono essere piccoli e semplici, più codice scrivi e più bug introduci.
- *Fail-safe default*: L'accesso deve basarsi sui permessi e non sulle esclusioni, se qualcosa va storto il sistema deve bloccarsi in uno stato sicuro negando l'accesso.
- *Complete Mediation*: Ogni singolo accesso a una risorsa deve essere verificato senza fidarsi ciecamente della cache.
- *Open Design*: La sicurezza non deve basarsi sulla segretezza del codice ma sulla forza dell'algoritmo o delle chiavi.
- *Separation of Privilege*: Richiedere più condizioni per sbloccare un'azione (2FA).
- *Least Privilege*: Utenti e processi devono avere solo i permessi strettamente necessari per fare il loro lavoro.
- *Layering*: Usare ostacoli multipli e sovrapposti, se l'attaccante buca il firewall deve trovare altri blocchi.

Superfici ed Alberi d'attacco

Per difendere un sistema è utile capire come possiamo attaccarlo, definiamo come **Attack Surface** l'insieme di tutte le vulnerabilità raggiungibili in un sistema. Si divide in:

- *Rete*: vulnerabilità dei protocolli di rete.
- *Software*: vulnerabilità nel codice di applicazioni, utility o sistemi operativi.
- *Umana*: si riferisce alle vulnerabilità create dal personale o da soggetti esterni.

Il nostro obiettivo è quello di ridurre il più possibile questa superficie, per analizzare le debolezze del sistema possiamo costruire un **albero di attacco** che rappresenta le minacce al nostro sistema:

- Il nodo *radice* è l'obiettivo dell'attaccante
- I *nodi intermedi* sono i sotto-obiettivi in AND o OR fra di loro
- Le *foglie* sono gli attacchi che può fare per colpire un sotto-obiettivo.

Questi alberi sono utilizzati dai team di sicurezza per capire qual è il percorso «più economico» o «più facile» per un attaccante, questo permette quindi di posizionare le difese (layering) dove sono più necessarie.

Computer Security Strategies

Come si gestisce quindi la sicurezza a livello aziendale?

- **Specification / Policy (Cosa Fare?)**: le regole che indicano cosa deve fare il sistema (es. lunghezza minima di una password). Queste devono bilanciare il costo della sicurezza col costo di un eventuale attacco. Deve prevedere almeno una descrizione informale del comportamento del sistema, facendo riferimento ai requisiti.
- **Implementation / Mechanisms (Come Farlo?)** - prevede quattro linee di azione *complementary*, ossia prevenzione, rilevamento, risposta e ripristino(backup).
- **Assurance ed Evaluation (Funziona?)**: dà la certezza che le misure di sicurezza installate facciano

esattamente quello che le regole (policy) richiedono. Questo si ottiene tramite test, certificazioni e analisi.

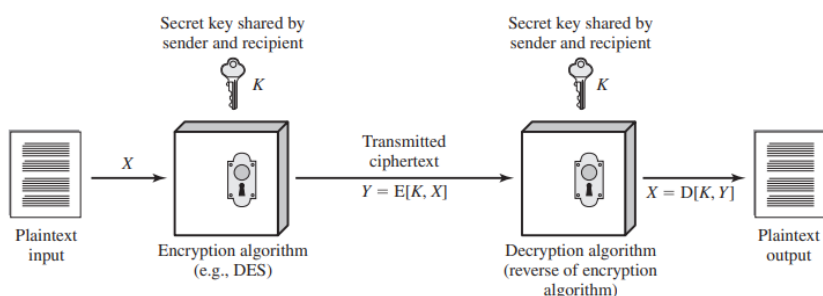
Strumenti di Crittografia

Crittografia Simmetrica

È la tecnica di crittografia universale per garantire la riservatezza dei dati. Il *mittente* e il *destinatario* per scambiarsi i messaggi devono condividere la stessa chiave segreta K .

Questa tecnica, per essere, sicura richiede:

- Un **algoritmo di crittografia** forte. L'avversario non dovrebbe essere in grado di decifrare il testo cifrato, o scoprire la chiave, anche se è in possesso di diversi testi cifrati insieme al testo in chiaro che li ha prodotti.
- Il **mittente ed il destinatario** devono aver ottenuto copie della chiave segreta in modo sicuro e devono custodirla in modo sicuro.



I *principali tipi di attacco* a queste tecniche sono:

- **Cryptanalysis**: sfrutta la natura matematica dell'algoritmo o se si conoscono alcune caratteristiche del testo non cifrato per dedurre la chiave.
- **Brute-Force**: si provano tutte le chiavi possibili finché non si ottiene un testo sensato. In media è necessario provare almeno la metà di tutte le chiavi possibili.

Tra i *principali algoritmi* per la crittografia simmetrica troviamo:

- **DES** (Data Encryption Standard): Utilizza blocchi cifrati, e non, da 64-bit e una chiave da 56-bit. Anche se è l'algoritmo più studiato la sua chiave da 56-bit lo rende molto debole contro i moderni processori di oggi.
- **3DES** (Triple DES): Ripete il DES tre volte usando 2 o 3 chiavi diverse, portando quindi la lunghezza effettiva della chiave a 112 o 168 bit. I problemi principali sono che è molto lento e usa ancora blocchi piccoli da 64-bit.
- **AES** (Advanced Encryption Standard): Creato per sostituire il 3DES. È molto più efficiente, utilizza blocchi da 128-bit e chiavi da 128, 192 o 256-bit.

Tipi di Crittografia Simmetrica

Ovviamente dobbiamo cifrare file più grandi di un qualsiasi blocco da 64 bit, per questo esistono diverse tipologie di cifratura:

- **Block Cipher**: cifra il testo elaborando un blocco alla volta, può riutilizzare le chiavi ed è il più comune.
- **Stream Cipher**: elabora gli elementi di input continuamente, producendo in output un elemento alla volta, ogni volta che si procede. Crittografa il testo in chiaro un byte alla volta ma può essere progettato per operare su un bit alla volta o su unità più grandi di un byte. Vengono usati per crittografare un flusso di dati come un canale di comunicazione o un server web.

Un metodo di cifratura che utilizza il cifrario a blocchi è il **ECB**:

- **Electronic Codebook (ECB)**: È la modalità base per usare i cifrari a blocchi su messaggi lunghi. Il testo in chiaro viene gestito b bit alla volta e ogni blocco viene crittografato usando la stessa chiave.

Autenticazione dei Messaggi e Hash Functions

⚡ La protezione contro gli attacchi attivi è nota come Autenticazione dei Messaggi.

Un messaggio, un file o una raccolta di dati si dice *autentica*, se non è stata alterata e proviene dalla sua presunta fonte. Inoltre, potremmo voler verificare che il messaggio non sia stato ritardato artificialmente.

Autenticazione senza riservatezza

Il mittente prende il messaggio in chiaro, usa una formula matematica e una chiave segreta per generare una stringa di caratteri (il *tag*). Poi invia il messaggio in chiaro insieme al tag. Garantisce al destinatario il messaggio l'hai mandato esattamente tu e che nessuno ha cancellato o aggiunto parole.

Autenticazione con riservatezza

L'autenticazione è fornita come funzione separata dalla crittografia. Questo significa che, di solito, si fanno due passaggi distinti:

1. Si genera il tag di autenticazione per il messaggio.
2. Si usa un algoritmo completamente diverso per "mescolare" e crittografare sia il messaggio che il tag.

Message Authentication Code (MAC)

Una delle principali tecniche di autenticazione prevede l'utilizzo di una chiave segreta per generare un piccolo blocco di dati, noto come **MAC**, che viene aggiunto al messaggio prima dell'invio.

Perché questo sistema sia applicabile, le due parti comunicanti (che chiameremo *A* e *B*) devono condividere a priori una chiave segreta, indicata con K_{AB} . Il processo si divide nei seguenti passaggi:

- **Generazione**: Quando *A* deve inviare un messaggio *M* a *B*, calcola il MAC applicando una funzione matematica complessa che prende in input sia il messaggio stesso che la chiave segreta. La formula è:

$$MAC = F(K_{AB}, M)$$

- **Trasmissione:** Il messaggio (in chiaro) e il MAC generato vengono inviati insieme al destinatario.
- **Verifica:** Una volta ricevuto il pacchetto, il destinatario B estrapola il messaggio in chiaro, recupera la sua copia della chiave segreta K_{AB} ed esegue lo stesso identico calcolo fatto dal mittente.
- **Confronto:** Infine, B confronta il MAC da lui calcolato con il MAC ricevuto in allegato al messaggio.

Se il confronto ha esito positivo, il destinatario ottiene tre importanti garanzie:

- **Integrità:** Il messaggio non è stato alterato o corrotto durante il tragitto.
- **Autenticazione:** Il messaggio proviene inequivocabilmente dal presunto mittente (poiché nessun altro, non conoscendo la chiave segreta, avrebbe potuto generare un MAC valido per quel messaggio).
- **Sequenza Corretta:** Se il messaggio include al suo interno un numero di sequenza, il destinatario può essere certo che l'ordine dei messaggi sia quello corretto e non abbia subito manomissioni.

Per generare questo codice di autenticazione si possono usare svariati algoritmi. Storicamente, uno dei metodi basati su cifratura simmetrica più noti prevede l'uso del **DES** (Data Encryption Standard): in sintesi, si cifra l'intero messaggio e si tengono solo gli ultimi bit del risultato cifrato (solitamente 16 o 32 bit) per usarli proprio come MAC.

One-Way Hash Function

Una funzione di hash crittografica (o *One-Way Hash Function*) è un'alternativa al MAC. A differenza di quest'ultimo, non accetta una chiave segreta come input, ma accetta solamente un messaggio di dimensione variabile M e produce in output un digest $H(M)$ di dimensione fissa.

Il Padding

In genere, il messaggio viene "paddato" (riempito) fino a raggiungere un multiplo intero di una lunghezza fissa (ad esempio 1024 bit). Per aumentare la sicurezza, il padding include sempre il valore della lunghezza del messaggio originale espresso in bit.

Esistono tre modi principali in cui un messaggio può essere autenticato utilizzando una funzione di hash:

1. **Digest crittografato con crittografia simmetrica:** Il digest generato viene cifrato usando una chiave segreta condivisa.
2. **Digest crittografato con crittografia a chiave pubblica:** Fornisce la firma digitale e l'autenticazione dei messaggi senza richiedere la distribuzione preventiva delle chiavi segrete alle parti comunicanti.
3. **Nessuna crittografia (Keyed Hash MAC):** Il mittente A e il destinatario B condividono una chiave segreta K , che viene incorporata direttamente nel processo di generazione dell'hash.
 - A calcola $MD_M = H(K \parallel M \parallel K)$ e invia a B il messaggio in chiaro concatenato al digest $[M \parallel MD_M]$.
 - B ricalcola il digest localmente e si assicura che sia uguale a quello ricevuto.

Perché l'ultimo approccio (Keyed Hash MAC) è il più comune?

- Il software di crittografia è generalmente lento.
- I costi dell'hardware dedicato alla crittografia non sono trascurabili.
- L'hardware di crittografia è ottimizzato per gestire grandi moli di dati, non piccoli digest.

- Gli algoritmi di crittografia tradizionali possono essere protetti da brevetto.

Proprietà di una Hash Function Sicura

Affinché una funzione di hash possa essere considerata sicura, deve rispettare i seguenti **6 requisiti fondamentali**:

1. Può essere applicata a blocchi di dati di *qualsiasi dimensione.
2. Produce un output di *lunghezza fissa.
3. $H(x)$ è **relativamente facile da calcolare** per qualsiasi dato x .
4. *One-Way (o Preimage Resistant)*: Per qualsiasi codice hash h dato, è computazionalmente impossibile risalire al messaggio originale, ovvero trovare un x tale che $H(x) = h$.
5. *Weak Collision Resistant (o Second Preimage Resistant)*: Dato un blocco specifico x , è computazionalmente impossibile trovare un blocco diverso y ($y \neq x$) che produca lo stesso hash, ovvero tale che $H(y) = H(x)$.
6. *Strong Collision Resistant (o Collision Resistant)*: È computazionalmente impossibile trovare una *qualsiasi* coppia di messaggi diversi (x, y) che abbiano lo stesso hash, ovvero tale che $H(x) = H(y)$.

Sicurezza e Algoritmi

⚠ Attacchi alle Funzioni di Hash I due approcci principali utilizzati per attaccare e violare una hash function sono la **crittoanalisi** e l'attacco tramite **brute-force**.

- **Algoritmo principale**: La famiglia di hash function più usata e sicura attualmente è il *Secure Hash Algorithm (SHA)*.
- **Altri utilizzi**: Oltre ad essere impiegate per l'autenticazione dei messaggi e per le firme digitali, le hash function sono ampiamente utilizzate per **cifrare le password** (salvandone solo l'hash nei database) o per i sistemi di rilevamento delle intrusioni.

Public-Key Encryption (Crittografia Asimmetrica)

Gli algoritmi a chiave pubblica si basano su funzioni matematiche piuttosto che su semplici operazioni su pattern di bit. Inoltre, la crittografia a chiave pubblica è **asimmetrica**, ovvero prevede l'uso di due chiavi separate:

- Una **pubblica** (resa nota e utilizzabile da altri).
- Una **privata** (nota solo al proprietario).

Queste chiavi sono selezionate in modo che, se una viene utilizzata per la cifratura, l'altra è usata per la decifratura.

In particolare, il funzionamento base per garantire la **Riservatezza** è il seguente:

1. Ogni utente genera una coppia di chiavi.
2. Ogni utente inserisce una delle due chiavi (quella pubblica) in un registro pubblico.
3. Se Bob vuole inviare un messaggio ad Alice, lo cifra usando la chiave pubblica di Alice.
4. Alice decifra il messaggio usando la sua chiave privata.

📄 Autenticazione e Integrità

Se invece si volesse garantire l'autenticazione e/o l'integrità dei dati, basta invertire l'uso delle chiavi: il mittente cifra il messaggio con la propria chiave *privata*, e il destinatario lo decifra con la chiave *pubblica* del mittente.

Requisiti per la Public-Key Encryption

Siano A e B i due comunicanti, PU_b la chiave pubblica di B , PR_b la chiave privata di B , M il messaggio e C il testo cifrato:

1. È computazionalmente facile per B generare una coppia di chiavi (PU_b, PR_b) .
2. È computazionalmente facile per A , conoscendo la chiave pubblica e il messaggio da cifrare, generare il testo cifrato: $C = E(PU_b, M)$.
3. È computazionalmente facile per B decifrare il testo cifrato risultante usando la chiave privata:
 $M = D(PR_b, C) = D(PR_b, E(PU_b, M))$.
4. È computazionalmente **impossibile**, conoscendo PU_b , determinare la chiave privata PR_b .
5. È computazionalmente **impossibile**, conoscendo PU_b e C , risalire al messaggio originale M .
6. Una delle due chiavi correlate viene usata per la cifratura, l'altra per la decifratura.

Algoritmi di Crittografia Asimmetrica

- **RSA (Rivest, Shamir, Adleman)**: È il più accettato e implementato; una dimensione della chiave di 1024 bit è considerata sufficientemente robusta.
- **Diffie-Hellman Key Agreement**: È il primo algoritmo a chiave pubblica ad essere stato pubblicato ed è limitato esclusivamente allo scambio delle chiavi.
- **Digital Signature Standard (DSS)**: Utilizza l'algoritmo SHA-1 e presenta una nuova tecnica di firma digitale, il Digital Signature Algorithm (DSA).
- **Elliptic Curve Cryptography (ECC)**: Sta prendendo piede perché offre la stessa sicurezza dell'RSA ma con una dimensione di bit molto più piccola.

Firma Digitale (Digital Signature)

La firma digitale serve a garantire tre cose fondamentali quando invii un messaggio

1. **Autenticazione**: Il messaggio l'hai mandato *sicuramente* tu.
2. **Integrità**: Il messaggio *non* è stato modificato durante il tragitto.
3. **Non ripudio**: Non puoi negare in futuro di aver mandato quel messaggio.

Per farla funzionare, si usa la crittografia asimmetrica (come l'RSA) ma **"al contrario"**. Invece di usare la chiave pubblica per cifrare (che garantisce la riservatezza), si usa la chiave privata.

Come funziona (Passo dopo passo)

Ipotizziamo che Alice voglia mandare un contratto (il messaggio M) a Bob e voglia firmarlo digitalmente.

Fase di Firma (Alice):

1. Alice prende il messaggio M e lo passa in una funzione di Hash (es. SHA-256) per ottenere un riassunto a lunghezza fissa, il digest: $h = H(M)$.
2. Alice cifra questo hash h usando la **sua chiave privata** (PR_a).

$$S = E(PR_a, h)$$

3. Il risultato S è la **Firma Digitale**. Alice invia a Bob il messaggio in chiaro allegandovi la firma: (M, S) .

Fase di Verifica (Bob):

1. Bob riceve il pacchetto (M, S) .
2. Bob prende il messaggio in chiaro M e ricalcola da solo l'hash: $h_1 = H(M)$.
3. Bob prende la firma S e la decifra usando la **chiave pubblica di Alice** (PU_a):

$$h_2 = D(PU_a, S)$$

4. Bob **confronta i due hash**. Se $h_1 == h_2$, la firma è valida.

❗ Perché è un sistema infallibile?

Se un hacker avesse modificato il messaggio in transito, l'hash h_1 calcolato da Bob sarebbe diverso da h_2 . Inoltre, poiché solo Alice possiede la chiave privata PR_a , nessun altro al mondo avrebbe potuto generare una firma S che si apre perfettamente con la chiave pubblica PU_a .

Public-Key Certificates (Certificati a Chiave Pubblica)

La firma digitale è fantastica, ma ha un tallone d'Achille enorme: il **problema della fiducia iniziale**.

Bob per verificare la firma di Alice deve usare la chiave pubblica di Alice (PU_a). Ma come fa Bob a essere **assolutamente certo** che quella chiave pubblica appartenga davvero ad Alice e non a un hacker (Darth) che si sta spacciando per lei?

Se Darth riesce a ingannare Bob facendogli scaricare la propria chiave pubblica dicendo "Ehi, sono Alice, questa è la mia chiave!", Darth potrà firmare tutti i documenti che vuole a nome di Alice. Questo si chiama attacco **Man-in-the-Middle**.

Per risolvere questo problema si usano i **Certificati a Chiave Pubblica**.

Come funziona (Il ruolo del Garante)

Il certificato introduce una "terza parte fidata", chiamata **CA (Certificate Authority)**. La CA è un ente autorevole (es. un governo, una banca o aziende informatiche certificate) di cui tutti si fidano ciecamente.

1. **La Richiesta**: Alice va dalla CA, mostra il suo documento di identità fisico (o altri metodi di autenticazione robusti) e consegna la sua chiave pubblica PU_a .
2. **La Creazione del Certificato**: La CA verifica l'identità di Alice, poi prende un documento digitale in cui scrive: *"Io, CA, certifico che la chiave pubblica PU_a appartiene alla persona chiamata Alice. Questo documento scade il 31 dicembre"*.

3. **La Firma della CA:** La CA **firma digitalmente** questo documento usando la **propria** chiave privata (PR_{ca}). Il documento risultante è il **Certificato**.
4. **L'Utilizzo:** Da questo momento, Alice non manda in giro la sua chiave pubblica "nuda". Invia invece a Bob il suo Certificato.
5. **La Verifica di Bob:** Bob riceve il certificato. Poiché la chiave pubblica della CA (PU_{ca}) è universale e preinstallata in tutti i computer e browser del mondo, Bob la usa per verificare la firma della CA sul certificato. Se la firma della CA è valida, Bob sa che le informazioni contenute nel documento sono vere, e quindi può fidarsi ciecamente di estrarre e usare la chiave pubblica PU_a di Alice per le comunicazioni successive.

Digital Envelopes (Buste Digitali)

La busta digitale viene usata per proteggere un messaggio garantendo sia velocità che sicurezza, senza dover prima fare in modo che mittente e destinatario condividano una chiave segreta:

1. Bob prepara un messaggio.
2. Genera una chiave simmetrica casuale che verrà usata una sola volta (chiave di sessione).
3. Cifra il messaggio intero tramite questa chiave monouso (sfruttando la velocità della crittografia simmetrica).
4. Cifra la chiave monouso tramite crittografia asimmetrica usando la **chiave pubblica di Alice**, la allega al messaggio cifrato e invia il tutto.

Numeri Random e Pseudo-Random

Diversi protocolli e algoritmi di sicurezza di rete necessitano di numeri casuali per funzionare correttamente. Ad esempio per:

- Generazione di chiavi per l'algoritmo RSA.
- Generazione di stream key per stream cipher simmetrici.
- Generazione di chiavi simmetriche temporanee/monouso (come nelle digital envelopes).
- Handshaking per prevenire attacchi di tipo replay.
- Generazione di chiavi di sessione.

Ci sono **due requisiti fondamentali** per una sequenza di numeri random:

1. **Casualità**, che viene a sua volta convalidata da due criteri statistici:
 - **Distribuzione uniforme:** la frequenza di occorrenza di ciascun numero dovrebbe essere approssimativamente la stessa.
 - **Indipendenza:** nessun valore nella sequenza può essere dedotto dagli altri.
2. **Imprevedibilità:** deve essere impossibile predire gli elementi futuri della sequenza basandosi sulla conoscenza degli elementi precedenti.

Random vs Pseudo-Random

Le applicazioni crittografiche in genere usano tecniche algoritmiche (software) per generare numeri casuali. Tuttavia, questi algoritmi sono di natura deterministica: producono sequenze di numeri che non sono *veramente* e statisticamente casuali, ma che sono calcolate in modo da superare i test di casualità. Questi numeri sono definiti **Pseudo-Random**.

Per ottenere numeri realmente casuali serve un **TRNG** (*True Random Number Generator*). Un TRNG utilizza una sorgente di entropia fisica e non deterministica. Ad esempio, la maggior parte di essi funziona misurando processi naturali altamente imprevedibili (es. rilevatori di impulsi di radiazioni ionizzanti o rumore termico).

Crittografia Simmetrica e Riservatezza dei messaggi

🔗 Principi di Crittografia Simmetrica

È detta **crittografia convenzionale**, crittografia a chiave segreta o (*secret-key*) o crittografia a chiave singola (*single-key*). Ha 5 componenti principali:

- **Plaintext**: Messaggio originale in input.
- **Encryption Algorithm**: Algoritmo che effettua varie trasformazioni sul plaintext.
- **Secret Key**: Che determina le trasformazioni effettuate dall'algoritmo.
- **Ciphertext**: Ovvero l'output dell'algoritmo (il testo cifrato).
- **Decryption Algorithm**: Ovvero l'inverso dell'encryption algorithm, usato per ritornare al testo in chiaro.

Sistemi di Crittografia

Sono meccanismi che utilizzano algoritmi, chiavi e protocolli per proteggere la confidenzialità, l'integrità, l'autenticazione e la non ripudiabilità.

Vengono classificati su 3 dimensioni:

- **Tipo di operazioni** utilizzate per cifrare il plain-text.
 - **Sostituzione**: Prende un insieme di byte e li rimpiazza con altri byte.
 - **Trasposizione**: I byte del testo vengono mischiati fra di loro.
Queste sono indipendenti dalla chiave e vengono ripetute un certo numero di volte (round) sul testo. Nell'effettuare queste operazioni non dobbiamo perdere informazione, questo significa che le operazioni devono essere invertibili.
- **Tipo di chiave** utilizzata: Simmetrica o asimmetrica
- Modo in cui viene trattato il **testo in chiaro**:
 - **Block**: Viene processato un blocco alla volta. Quindi da un blocco in input otteniamo un blocco in output.
 - **Stream**: ogni elemento viene elaborato uno alla volta in modo continuo

(Cifrari a Blocchi) Block Cipher

Adesso vedremo alcuni algoritmi e patter nel cifrario a blocchi.

Struttura del Cifrario di Feistel

Si tratta di una struttura utilizzata da molti algoritmi di *crittografia simmetrica a blocchi*.

📄 Funzionamento

1.Divisione in due metà:

Il blocco di dati originale (es. 64 bit) viene spaccato esattamente a metà: una parte Sinistra (L_0) e una parte Destra (R_0).

2.L'elaborazione (Round i):

La funzione F elabora solo metà dei dati.

In ogni singolo round, avvengono due cose simultaneamente:

- La vecchia metà Destra (R_{i-1}) **diventa automaticamente** la nuova metà Sinistra (L_i) senza subire alcuna modifica.
- La vecchia metà Sinistra (L_{i-1}) viene "mescolata" usando l'operatore matematico XOR ($\oplus$) con il risultato di una funzione F . Questa funzione F prende in input la vecchia metà Destra e la sottochiave segreta di quel round (K_i).

3.Lo Scambio (Swap):

Le due nuove metà ottenute vengono scambiate di posto, pronte per entrare nel round successivo.

4.Ricomposizione Finale:

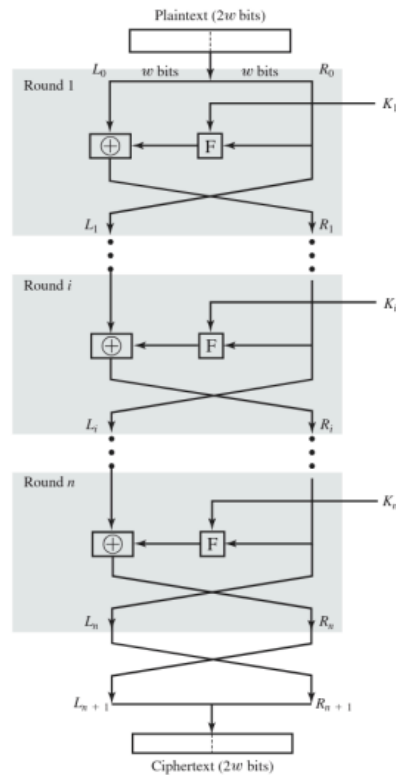
Al termine di tutti i round previsti (es. dopo 16 cicli), l'ultimo scambio viene saltato e le due metà vengono riunite per formare il testo cifrato finale.

Se lo scriviamo in formule (molto utili per i tuoi appunti di informatica), un round i si calcola così:

$$\begin{aligned} L_i &= R_{i-1} \\ R_i &= L_{i-1} \oplus F(R_{i-1}, K_i) \end{aligned}$$

L'esatta realizzazione di un cifrario a blocchi simmetrico dipende da svariati parametri:

- **Dimensione del blocco:** Più è grande più è sicuro ma lento (di solito si usano blocchi da 128 bit).
- **Dimensione della chiave:** Più è grande più è sicuro ma lento (di solito 128 bit).
- **Numero di round:** Più round garantiscono più sicurezza (di solito sono 16).
- **Algoritmo di generazione della sottochiave:** Più è complesso, più è difficile la crittoanalisi.
- **Funzione di round:** Più è complessa, più è difficile la crittoanalisi.



Inoltre, altri due aspetti fondamentali da considerare durante la progettazione sono:

- **Cifratura / Decifratura veloce:** La velocità di esecuzione dell'algoritmo è un fattore critico per le performance del sistema.
- **Facilità di analisi:** se l'algoritmo può essere spiegato in modo conciso e chiaro è più facile analizzarlo ed individuare le criticità.

Data Encryption Standard (DES)

Si basa sul **Data Encryption Algorithm (DEA)**, che può essere descritto come segue: il testo in chiaro ha una lunghezza di 64 bit e la chiave di 56 bit. È una variante del cifrario di Feistel in cui sono previsti 16 cicli (e quindi 16 sottochiavi). Per la decifratura basta usare le chiavi in ordine inverso.

Triple DES (3DES)

Utilizza tre chiavi e tre esecuzioni dell'algoritmo DES. La funzione segue una sequenza di Cifratura-Decifratura-Cifratura:

$$C = E(K_3, D(K_2, E(K_1, P)))$$

Dove P è il plaintext, E è la cifratura e D la decifratura.

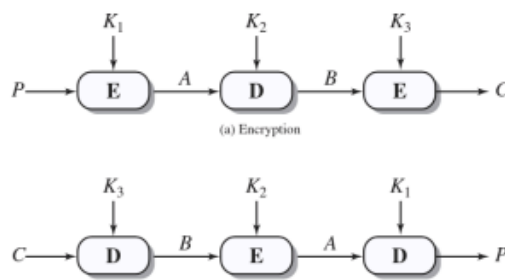
Per la decifratura basta invertire il tutto:

$$P = D(K_1, E(K_2, D(K_3, C)))$$

Non c'è motivazione crittografica nell'usare la decifratura al secondo step, l'unico vantaggio è la retrocompatibilità col DES se $K_1 = K_2 = K_3$.

$$C = E(K_3, D(K_2, E(K_1, P))) = E[K, P]$$

Con tre chiavi distinte il 3DES ha una lunghezza di chiave di 168 bit. Se $K_1 = K_3$ allora sarà di 112 bit.



Esempio Pratico: Perché Cifrare-Decifrare-Cifrare?

Immagina il 3DES come una cassaforte dentro un'altra cassaforte dentro un'altra ancora. Perché il passaggio in mezzo è una "decifratura"? Se un vecchio sistema supporta solo il DES normale (1 singola chiave), possiamo impostare K_1 , K_2 e K_3 in modo che siano tutte uguali alla stessa chiave. In questo modo: cifro, decifro (annullando la prima cifratura) e poi cifro di nuovo. Il risultato netto è una singola cifratura DES, rendendo il 3DES compatibile con i vecchi sistemi!

Advanced Encryption Standard (AES)

L'**algoritmo AES** elabora i dati a "blocchi" di 128 bit (16 byte). Questi 16 byte vengono disposti in una griglia 4x4, chiamata **State Array**. Tutte le operazioni di cifratura non fanno altro che manipolare matematicamente i byte all'interno di questa griglia.

Per cifrare, l'**AES** utilizza una chiave segreta iniziale da 128, 192 o 256 bit. Poiché l'algoritmo si ripete in più cicli (o round), una singola chiave non è sufficiente. La chiave di partenza viene quindi "espansa" in una sequenza di Word, dove ogni **Word** equivale a 4 byte, per generare materiale crittografico aggiuntivo. Ad ogni round, l'algoritmo preleva una porzione di questa chiave espansa per formare una nuova griglia 4x4 da 16 byte. Questa matrice prende il nome di **Round Key** e viene combinata direttamente con lo State Array.

Per generare la chiave partendo dalle 4 word iniziali: $i = i-1 \ i-4$

Punti importanti riguardo AES:

1. Non è una struttura Feistel, ma elabora l'intero blocco di dati in parallelo durante ogni round usando sostituzioni e permutazioni.
2. La chiave viene espansa in un array di 44 words da 32-bit. Quattro words distinte formano la **round key** per un round.
3. I **round sono 10** ed ogni round è composto da 4 fasi, una di permutazione e tre di sostituzione:
 - **Sostituzione dei byte (SubBytes)**: L'algoritmo prende ogni singolo byte della matrice 4x4 e lo sostituisce con un altro byte, andandolo a cercare in una tabella precompilata (la S-Box).

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

- Se nel byte c'è scritto "A5", guardo la tabella, trovo "A5" e lo sostituisco con "C2".
 - **Shift delle righe (ShiftRows):** prende le 4 righe della matrice e le fa scorrere (shift) verso sinistra in modo circolare:
 - Riga 0: Non viene toccata.
 - Riga 1: Scorporata e spostata a sinistra di 1 posizione.
 - Riga 2: Spostata a sinistra di 2 posizioni.
 - Riga 3: Spostata a sinistra di 3 posizioni.
 In questo modo, i byte che prima erano vicini nella stessa colonna vengono sparpagliati su colonne diverse.
 - **Mescolamento delle colonne (MixColumns):** Sostituzione di ogni byte di una colonna in funzione di tutti i byte della colonna stessa.
 - **Aggiunta della round key (AddRoundKey):** In questo ultimo passaggio, si prende una porzione della chiave segreta espansa (la Round Key, anch'essa una matrice 4x4) e la si fonde con la matrice di Stato facendo un semplice XOR bit a bit.
4. La fase di aggiunta della round-key è l'unica ad usare la chiave; per questo motivo il cifrario inizia e finisce con questa fase. Le altre fasi mescolano i bit ma da sole non fornirebbero sicurezza.

Modalità di funzionamento dei Block Ciphers

Le **modalità di funzionamento** sono le regole che ci dicono **come utilizzare gli algoritmi più e più volte** per cifrare un file gigante, blocco dopo blocco. Questo perché, gli algoritmi da soli sono vulnerabili in modo differente, e con il metodo d'uso le risolvo.

⚡ L'algoritmo spiega come cifrare un blocco, mentre la modalità spiega come suddividere il messaggio da cifrare in blocchi

Electronic Codebook Mode (ECB)

Metodo più semplice: il testo viene elaborato b bit alla volta e ogni blocco viene cifrato con la stessa chiave. Se lo stesso blocco compare più di una volta nel messaggio, produce lo stesso testo cifrato.

Metodo debole contro la crittoanalisi.

Cipher Block Chaining Mode (CBC)

L'input dell'algoritmo è lo XOR tra il blocco in chiaro corrente e il blocco cifrato precedente:

$$C_j = E(K, [C_{j-1} \ P_j])$$

Per la decifratura, ogni blocco cifrato viene prima elaborato dall'algoritmo per poi essere messo in XOR col testo cifrato precedente:

$$D(K, C_j) = C_{j-1} \ P_j$$

$$C_{j-1} \ D(K, C_j) = P_j$$

Per il primo blocco si usa un Vettore di Inizializzazione (IV) protetto.

Cipher Feedback Mode (CFB)

Un Block Cipher (come l'AES) elabora i dati a blocchi di dimensione fissa, diciamo b bit (es. 128 bit). Se tu vuoi inviare solo un carattere alla volta (che pesa s bit, es. 8 bit), l'AES non te lo permette: ti obbliga ad aggiungere 120 bit di spazzatura (il *padding*) per riempire il blocco, sprecando banda e tempo.

Il CFB elimina questo problema trasformando il blocco in un flusso continuo.

Gli "Attori" in gioco

- b : La dimensione del blocco intero (es. 128 bit).
- s : L'unità di trasmissione, ovvero il pezzettino piccolo che vuoi spedire adesso (es. 8 bit).
- **Registro a scorrimento**: Una "scatola di memoria" lunga b bit. All'inizio viene riempita con un Vettore di Inizializzazione (**IV**), un numero di partenza condiviso.
- S_s : Un selettore che significa "prendi solo gli s bit più a sinistra" e ignora tutto il resto.

La Cifratura

Immagina di dover cifrare il primo pezzettino del tuo messaggio (P_1).

$$C_1 = P_1 \oplus S_s[E(K, IV)]$$

Ecco cosa fa fisicamente il computer:

1. $E(K, IV)$: L'algoritmo prende l'intero registro (che ora contiene l'IV) e lo cifra con la chiave K . Il risultato è un blocco di rumore lungo b bit.
2. $S_s[\dots]$: Dato che a noi serve cifrare solo s bit, prendiamo questo blocco di rumore e ne *tagliamo via un pezzo*, tenendo solo gli s bit più a sinistra (i più significativi).
3. **(XOR)**: Prendiamo il nostro piccolo testo in chiaro P_1 (da s bit) e lo mettiamo in XOR con il rumore appena tagliato.
4. **Risultato**: Otteniamo C_1 , il nostro testo cifrato da s bit, pronto per essere spedito senza alcun padding!

Il Feedback

Per cifrare il secondo pezzettino (P_2), non possiamo riusare lo stesso registro di prima, altrimenti creeremmo lo stesso rumore. Dobbiamo aggiornarlo.

- I vecchi bit nel registro slittano a sinistra. I primi s bit "cadono" fuori e vengono buttati.
- Si crea uno spazio vuoto a destra. In quello spazio viene infilato proprio C_1 (il testo cifrato appena calcolato). Il registro è ora aggiornato e pronto per essere cifrato (E) nel ciclo successivo.

La Decifratura

La formula della decifratura è identica, ma invertita ai lati:

$$P_1 = C_1 \oplus S_s[E(K, IV)]$$

La cosa che confonde spesso gli studenti è: **perché usiamo E (Cifratura) per decifrare?**

La risposta sta nell'operatore logico XOR.

Il destinatario non deve decifrare il testo, deve *ricreare lo stesso identico rumore* che hai generato tu. Quindi, il destinatario prende il suo registro (che inizia con l'IV), *lo cifra* con la funzione *E* per generare lo stesso rumore, ne prende gli *s* bit più a sinistra e fa lo XOR con il pacchetto cifrato C_1 appena ricevuto.

Counter Mode (CTR)

Struttura del Contatore

Per ogni blocco del messaggio viene generato un **Contatore** indipendente (grande quanto il blocco che richiede l'algoritmo). Di solito è suddiviso in due parti:

- **Nonce (Number used once):** Un valore univoco casuale (o progressivo) per quel messaggio. Garantisce che messaggi diversi non usino mai la stessa sequenza.
- **Counter:** Un numero intero che parte da un valore base (es. 0) e si incrementa rigorosamente di 1 per ogni nuovo blocco del messaggio.

Fase di Cifratura

L'obiettivo è creare una maschera di bit (Keystream) da sovrapporre al messaggio.

1. **Generazione del Keystream:** Il contatore del blocco corrente ($Contatore_i$) viene cifrato passando attraverso l'algoritmo insieme alla Chiave Segreta (K).
2. **Operazione XOR:** Il blocco di testo in chiaro (P_i) viene messo in XOR () bit a bit con il Keystream appena generato per produrre il blocco cifrato (C_i).

Formula di Cifratura:

$$C_i = P_i \oplus E_K(Contatore_i)$$

Fase di Decifratura

L'obiettivo della decifratura è semplicemente ricreare l'esatto Keystream generato in andata, per poi annullare l'effetto dello XOR.

1. Si prende il $Contatore_i$ (che è noto/deducibile) e lo si **cifra** con la chiave K . L'AES restituisce l'identico Keystream.
2. Si esegue lo XOR tra il testo cifrato (C_i) e il Keystream. Grazie alle proprietà matematiche dello XOR, i bit confusi si annullano, restituendo il plaintext.

Formula di Decifratura:

$$P_i = C_i \oplus E_K(Contatore_i)$$

I principali vantaggi di questa modalità sono:

- **Efficienza Hardware:** Può essere eseguita in parallelo a differenza delle altre modalità, poiché non è necessario completare un blocco prima di passare al successivo.
- **Efficienza Software:** Grazie all'opportunità di esecuzione parallela è possibile utilizzare efficacemente processori che supportano funzionalità parallele.

- **Pre-elaborazione:** L'esecuzione dell'algoritmo di cifratura non dipende dall'input del testo, pertanto è possibile effettuare una pre-elaborazione della cifratura del counter, per poi fare solo gli XOR.
- **Accesso Casuale:** L' i -esimo blocco può essere elaborato senza calcolare i precedenti.
- **Sicurezza Dimostrabile:** È sicuro almeno quanto le altre modalità.
- **Semplicità:** Richiede solo l'implementazione dell'algoritmo di cifratura e non di quello di decifratura.

Stream Cipher

Gli stream ciphers elaborano gli elementi in input in modo continuo, producendo l'output un elemento alla volta. Tipicamente viene cifrato un byte alla volta. In questa struttura, una chiave viene immessa in un generatore pseudo-casuale che produce un flusso di numeri a 8 bit, detto **flusso di chiavi (keystream)**. Questo viene combinato con il testo in chiaro tramite l'operazione XOR bit a bit.

RC4

È uno stream cipher a chiave variabile con operazioni orientate ai byte, basato sull'uso di una permutazione casuale con un periodo che è maggiore di 10^{100} con estrema probabilità. Per ogni byte di output sono richieste solo tra le 8 e le 16 operazioni macchina, rendendo il cifrario molto veloce.

Una chiave a lunghezza variabile (1-256 byte) viene usata per inizializzare un vettore S di 256 byte. In ogni momento, S contiene una permutazione di tutti i numeri a 8 bit da 0 a 255. Per la cifratura/decifratura, un byte K viene generato da S e man mano che ciò accade le voci in S vengono permutate.

Inizializzazione di S

Per iniziare, S viene riempito con i numeri da 0 a 255 in ordine crescente. Poi si crea un vettore T da 256 byte riempito con la chiave ripetuta. T viene quindi usato per produrre la permutazione di S :

$$\forall i \text{ si fa lo swap di } (S[i], S[j]) \text{ tale che } j = (j + S[i] + T[i]) \bmod 256$$

Stream Generation

Prevede di scorrere S e scambiare ogni $S[i]$ con un $S[j]$ tale che:

$$j = (j + S[i]) \bmod 256$$

Ad ogni iterazione prende anche il valore $k = S[t]$ tale che:

$$t = (S[i] + S[j]) \bmod 256$$

da mettere in XOR col prossimo byte del testo cifrato/decifrato.

CODICE

```
/* Initialization */
for i = 0 to 255 do
    S[i] = i;
```

```
- [ ] T[i] = K[i mod keylen];
```

```
/* Initial Permutation of S */
```

```
j = 0;
```

```
for i = 0 to 255 do
```

```
    j = (j + S[i] + T[i]) mod 256;
```

```
    Swap (S[i], S[j]);
```

```
/* Stream Generation */
```

```
i, j = 0;
```

```
while (true)
```

```
    i = (i + 1) mod 256;
```

```
    j = (j + S[i]) mod 256;
```

```
    Swap (S[i], S[j]);
```

```
    t = (S[i] + S[j]) mod 256;
```

```
    k = S[t];
```

☰ Esempio Pratico: Lo XOR negli Stream Cipher

L'operazione XOR (OR esclusivo) è magica perché è perfettamente reversibile. Regola base: 1 XOR 1 = 0; 0 XOR 0 = 0; 1 XOR 0 = 1.

- **Plaintext:** 11001100
- **Keystream:** 01101100
- **Ciphertext (XOR):** 10100000 (Questo è ciò che viaggia in rete) Quando il destinatario riceve il ciphertext, riapplica lo stesso identico keystream con un altro XOR e riottiene magicamente il Plaintext originale!

Gestione e Sicurezza

Crittoanalisi

È il processo che tenta di scoprire il testo in chiaro o la chiave tramite una strategia che si basa sulla natura dello schema di cifratura e sulle informazioni a disposizione del crittoanalista.

Presumendo che l'avversario conosca sempre l'algoritmo di cifratura usato (anche se non sempre è così), si presentano varie situazioni in base alle informazioni disponibili:

- **Ciphertext-only attack (Solo testo cifrato):** Se l'attaccante conosce solo il testo cifrato, può tentare un attacco a forza bruta (*bruteforce*) oppure analizzare il testo cifrato applicando test statistici per cercare di individuare regolarità.
- **Known-plaintext attack (Testo in chiaro noto):** Se conosce uno o più messaggi in chiaro con le relative crittografie, oppure sa che determinati pattern di testo in chiaro appariranno sicuramente in un messaggio, può dedurre la chiave in base al modo in cui il testo in chiaro noto viene trasformato.
- **Chosen-plaintext attack (Testo in chiaro scelto):** Se può ottenere o accedere al sistema sorgente per inserire un messaggio scelto a sua preferenza, può cifrare tale messaggio con uno schema utile a rivelare la struttura della chiave.

- **Chosen-ciphertext attack (Testo cifrato scelto):** Se ha accesso al sistema e può scegliere il testo cifrato da far decifrare per analizzare il testo in chiaro risultante (o se ha a disposizione coppie di chiaro/cifrato mirate).

📌 Principio di Sicurezza Generale

In generale, una cifratura è considerata sicura se:

1. Il costo economico o computazionale necessario per la decifratura supera il valore effettivo delle informazioni protette.
2. Il tempo necessario per la decifratura supera la durata utile dell'informazione stessa (ovvero, l'informazione diventa obsoleta prima che l'attaccante riesca a decifrarla).

Fasi di un Attacco

1. Ricognizione (Reconnaissance)

È la fase di studio del bersaglio. L'attaccante raccoglie il maggior numero di informazioni possibili sull'obiettivo senza farsi notare (o facendosi notare il meno possibile).

- **Ricognizione Passiva:** Raccolta di dati pubblici (OSINT - Open Source Intelligence). Ricerca di dipendenti su LinkedIn per futuri attacchi phishing, analisi dei record DNS, studio dell'infrastruttura aziendale tramite motori di ricerca.
- **Ricognizione Attiva:** Interazione diretta, ma leggera, con i sistemi dell'obiettivo (es. interrogazioni dei server per capire quali indirizzi IP sono attivi).

2. Scansione e Enumerazione (Scanning & Enumeration)

L'attaccante ha la mappa dell'edificio e ora cerca la "finestra aperta". In questa fase si usano strumenti automatizzati (come *Nmap* o scanner di vulnerabilità) per sondare attivamente le difese.

- **Cosa si cerca:** Porte aperte, servizi in esecuzione, versioni specifiche dei sistemi operativi e dei software.
- **L'obiettivo:** Trovare una vulnerabilità nota (una falla nel software, una configurazione errata, una password di default non cambiata) associata a uno di quei servizi.

3. Ottenimento dell'Accesso (Gaining Access / Exploitation)

È l'effrazione vera e propria. L'attaccante sfrutta la vulnerabilità trovata nella fase precedente per penetrare nel sistema.

- **L'attacco:** Può essere l'invio di un *exploit* che manda in crash un servizio (es. Buffer Overflow), un attacco di *Phishing* in cui un dipendente clicca su un link malevolo, o un attacco *Brute Force* per indovinare una password.
- **Privilege Escalation:** Spesso l'accesso iniziale garantisce solo poteri limitati (utente standard). L'attaccante dovrà sfruttare ulteriori vulnerabilità interne per "scalare i privilegi" e diventare

Amministratore (Admin/Root).

4. Mantenimento dell'Accesso (Maintaining Access)

Una volta dentro, l'attaccante non vuole rischiare di perdere il controllo se il server viene riavviato o se la password dell'utente compromesso viene cambiata. Deve lasciare la "porta sul retro" aperta.

- **Gli strumenti:** Vengono installate *Backdoor*, *Rootkit* o *Trojan*. Spesso vengono creati nuovi account amministrativi nascosti o vengono alterati i file di avvio del sistema operativo per garantire che il malware parta in automatico.
- **L'obiettivo:** Garantirsi la persistenza (Poter entrare e uscire a piacimento nei giorni o mesi successivi).

5. Copertura delle Tracce (Covering Tracks)

Il ladro è dentro, ha il controllo, ma non vuole far scattare gli allarmi o lasciare impronte digitali che possano portare a lui in caso di indagini forensi.

- **Le azioni:** L'attaccante cancella o altera i log di sistema (i registri che annotano chi entra e cosa fa), cancella la cronologia dei comandi (es. il file `.bash_history` su Linux) e nasconde i propri file malevoli camuffandoli da processi di sistema legittimi.

📌 **Azione sugli Obiettivi (Actions on Objectives)** Nei modelli moderni, si aggiunge sempre quest'ultima fase pratica. Una volta completati i 5 passi, l'hacker fa ciò per cui è entrato: **esfiltra** (ruba) dati sensibili, distrugge i server, o installa un **Ransomware** per cifrare tutto il disco rigido e chiedere un riscatto pecuniario.

Distribuzione delle Chiavi

Affinché la crittografia simmetrica funzioni, le due parti coinvolte devono condividere la stessa chiave proteggendola da terzi e, possibilmente, cambiandola frequentemente. Pertanto è fondamentale la tecnica di distribuzione della chiave, la quale può essere realizzata in diversi modi:

1. *A* potrebbe selezionare una chiave e consegnarla fisicamente a *B*.
2. Una terza parte fidata potrebbe selezionare una chiave e consegnarla fisicamente ad *A* e *B*.
3. Se *A* e *B* hanno precedentemente e recentemente usato una chiave, *A* può cifrare la nuova chiave usando la vecchia e trasmetterla a *B*.
4. Se *A* e *B* hanno una connessione cifrata con una terza parte *C* (il KDC), *C* potrebbe consegnare una chiave tramite tali connessioni ad *A* e *B*.

In particolare, per la quarta opzione si distingue tra due tipi di chiavi:

- **Chiave di sessione (Session Key):** Quando due terminali vogliono comunicare, stabiliscono una connessione logica temporanea, durante la quale cifrano i dati con una chiave di sessione monouso, che poi viene distrutta al termine della comunicazione.
- **Chiave permanente (Permanent Key):** È una chiave a lungo termine usata tra le entità e il centro di distribuzione esclusivamente per distribuire in modo sicuro le chiavi di sessione.

Elementi fondamentali dell'architettura:

- **KDC (Centro di Distribuzione delle Chiavi):** Determina quali sistemi sono autorizzati a comunicare tra loro e fornisce loro una chiave di sessione temporanea.
- **SSM (Modulo di Servizio di Sicurezza):** Esegue materialmente la crittografia end-to-end e ottiene le chiavi di sessione dal KDC per conto degli utenti/applicazioni.

☰ Esempio Pratico: KDC e Chiavi di Sessione

Immagina il KDC come la reception di un albergo super sicuro. Alice e Bob lavorano nello stesso edificio ma in reparti diversi, e le loro porte sono chiuse. Alice vuole parlare con Bob. Va alla reception (KDC) usando il suo badge aziendale (chiave permanente). La reception le dà un pass temporaneo valido solo per 1 ora (Chiave di sessione) che apre una specifica stanza riunioni. Finito il meeting, il pass viene distrutto. Se qualcuno ruba quel pass il giorno dopo, è del tutto inutile.

Crittografia Asimmetrica e Autenticazione dei Messaggi

Funzioni di Hash Semplici

📄 Concetto Base

Nelle funzioni di hash, il messaggio in input viene suddiviso in una sequenza di **blocchi di dimensione fissa** (n bit). Questi blocchi vengono elaborati uno alla volta (in modo iterativo) per produrre in output un valore finale di lunghezza fissa (chiamato **Digest** o valore di hash), anch'esso tipicamente di n bit.

Lo XOR Bit-a-Bit (Bitwise XOR)

Una delle funzioni di hash più elementari consiste nell'eseguire lo XOR logico ($\oplus$) di tutti i blocchi del messaggio.

Se abbiamo un messaggio diviso in N blocchi, il bit in posizione i del nostro hash finale (C_i) è semplicemente lo XOR dell' i -esimo bit di tutti i blocchi:

$$C_i = b_{i1} \oplus b_{i2} \oplus \dots \oplus b_{iN}$$

(Dove b_{ij} è l' i -esimo bit del j -esimo blocco).

- **Efficacia:** Questa operazione genera un "bit di parità" per ogni colonna. È un metodo ottimo per controllare l'integrità di **dati casuali** (la probabilità che l'hash resti invariato a fronte di un errore nel file originale è di appena 2^{-n}).
- **Il Limite:** È molto inefficace se i dati sono formattati in modo prevedibile (come testi scritti o file strutturati), perché le regolarità del testo non vengono mascherate.

Il Miglioramento: Shift Circolare (Rotazione)

Per mitigare il problema dei dati prevedibili, si introduce una semplice operazione: **eseguire uno shift circolare (rotazione) di 1 bit** sul valore di hash parziale subito dopo aver elaborato ogni blocco, prima di

fare lo XOR con il blocco successivo.

Questa piccola rotazione rimescola continuamente i bit, randomizzando maggiormente l'input e distruggendo le regolarità strutturali del messaggio in chiaro.

Vulnerabilità (Attacco del Blocco Spazzatura)

⚡ Inutile per la Sicurezza

Sebbene lo XOR e lo shift siano veloci ed efficienti per rilevare *errori accidentali*, non offrono alcuna resistenza contro alterazioni *intenzionali*.

Se un attaccante vuole manomettere il testo in chiaro, può farlo facilmente. Essendo lo XOR un'operazione completamente reversibile e lineare, l'attaccante può sostituire il messaggio originale con uno maligno e aggiungere alla fine un **blocco "spazzatura"** (garbage block) calcolato appositamente. Questo blocco fa sì che lo XOR finale coincida esattamente con l'hash del messaggio originale, ingannando i controlli.

Tentativo (Fallito) di Mitigazione tramite Modalità CBC

Per impedire la manomissione del testo in chiaro, un'idea storica è stata quella di calcolare l'hash tramite XOR, aggiungerlo alla fine del file in chiaro, e poi **cifrare l'intero pacchetto utilizzando un cifrario a blocchi in Modalità CBC**.

Siano X i blocchi in chiaro (incluso l'hash finale come X_{N+1}) e Y i blocchi cifrati. Le formule di decifratura (utilizzando la chiave K e il vettore IV) sarebbero:

$$X_1 = IV \oplus D_K(Y_1)$$

$$X_i = Y_{i-1} \oplus D_K(Y_i)$$

$$X_{N+1} = Y_N \oplus D_K(Y_{N+1})$$

Sostituendo le variabili, l'hash finale decifrato (X_{N+1}) risulta essere:

$$X_{N+1} = X_1 \oplus X_2 \oplus \dots \oplus X_N = [IV \oplus D_K(Y_1)] \oplus [Y_1 \oplus D_K(Y_2)] \oplus \dots \oplus [Y_{N-1} \oplus D_K(Y_N)]$$

Perché questa mitigazione fallisce? Il valore dell'hash decifrato (X_{N+1}) dipende interamente dai blocchi cifrati (Y) e dalle operazioni XOR, che sono lineari e pubbliche. Anche senza conoscere la chiave K , un attaccante matematicamente preparato può sfruttare la struttura del CBC per manipolare i blocchi ciphertext Y . Può iniettare modifiche tali per cui, durante la decifratura, le alterazioni si cancellino a vicenda nell'ultimo blocco, risultando in un falso messaggio in chiaro il cui hash risulta però perfettamente coerente con i dati.

SHA (Secure Hash Algorithm)

SHA-1 (L'algoritmo storico)

- **Output:** Produce un valore di hash di **160 bit** (20 byte).
- **Elaborazione:** Lavora su blocchi di input da 512 bit.

- **Stato attuale:** Deprecato e Insicuro

SHA-1 è stato lo standard di fatto per oltre un decennio (usato nei certificati SSL, in Git, ecc.), ma oggi è considerato **crittograficamente rotto**. A causa delle dimensioni ridotte del suo hash e di debolezze strutturali, è vulnerabile agli **attacchi di collisione** (ovvero la capacità di creare due file diversi che generano lo stesso identico hash). Nel 2017 è stata dimostrata la prima collisione pratica su SHA-1.

SHA-2 (Lo standard moderno)

Per superare i limiti del suo predecessore, è stata introdotta la famiglia **SHA-2**. Non si tratta di un singolo algoritmo, ma di una suite di funzioni che prendono il nome dalla lunghezza del *digest* che producono. Le versioni principali sono **SHA-256**, **SHA-384** e **SHA-512**.

- **Struttura:** SHA-2 mantiene la stessa architettura di base (Merkle-Damgård) e utilizza le stesse operazioni logiche elementari di SHA-1 (XOR, AND, OR, Shift, Rotazioni).
- **Miglioramenti:** Nonostante la somiglianza strutturale, SHA-2 applica queste operazioni in modo molto più complesso, introducendo costanti diverse e un *message schedule* (espansione del messaggio) molto più robusto.
- **Sicurezza:** A oggi, la famiglia SHA-2 non presenta vulnerabilità pratiche ed è lo standard crittografico raccomandato a livello globale.

SHA-512

📘 Specifiche Tecniche

- **Input Max:** Lunghezza del messaggio $< 2^{128}$ bit.
- **Dimensione Blocco:** 1024 bit.
- **Dimensione Digest (Output):** 512 bit.
- **Word Size:** Le operazioni matematiche vengono eseguite su *word* da 64 bit.

L'elaborazione del messaggio avviene elaborando un blocco da 1024 bit alla volta, seguendo questi 5 step sequenziali:

- **Aggiunta dei Bit di Riempimento (Padding)**

Lo scopo del padding è fare in modo che la lunghezza del messaggio sia esattamente a 128 bit di distanza dal prossimo multiplo di 1024.

- Il messaggio originario di lunghezza L viene riempito in modo che la nuova lunghezza soddisfi la congruenza: $L \pmod{1024} = 86$.
- **Come avviene:** Il padding viene *sempre* aggiunto (anche se il messaggio era già della lunghezza "giusta"). Consiste sempre in un singolo bit **1** seguito dal numero necessario di bit **0** (da un minimo di 1 a un massimo di 1024 bit di riempimento).

- **Aggiunta della Lunghezza (Length Append)**

Al messaggio paddato (che ora termina al bit 896 di un blocco) viene accodato un blocco finale di **128 bit**.

- Questo blocco viene trattato come un intero senza segno in formato *Big-Endian* e contiene l'esatta lunghezza L del messaggio originale (prima del padding).
- A questo punto, l'intero messaggio ha una lunghezza che è un **multiplo esatto di 1024 bit**, pronto per essere diviso in blocchi.

• Inizializzazione del Buffer (Hash State)

Per memorizzare i risultati intermedi e il digest finale, viene utilizzato un buffer di memoria da 512 bit, suddiviso in **8 registri da 64 bit** (spesso chiamati a, b, c, d, e, f, g, h).

- **Valori Iniziali (IV):** Questi 8 registri vengono inizializzati con i primi 64 bit delle parti frazionarie delle radici quadrate dei **primi 8 numeri primi** (2, 3, 5, 7, 11, 13, 17, 19).
- Il formato di memorizzazione è sempre *Big-Endian*.

• Elaborazione dei Blocchi (Compression Function)

Questo è il "cuore" matematico dell'algoritmo. L'elaborazione iterativa riceve in input lo stato attuale del buffer e il blocco corrente del messaggio da 1024 bit.

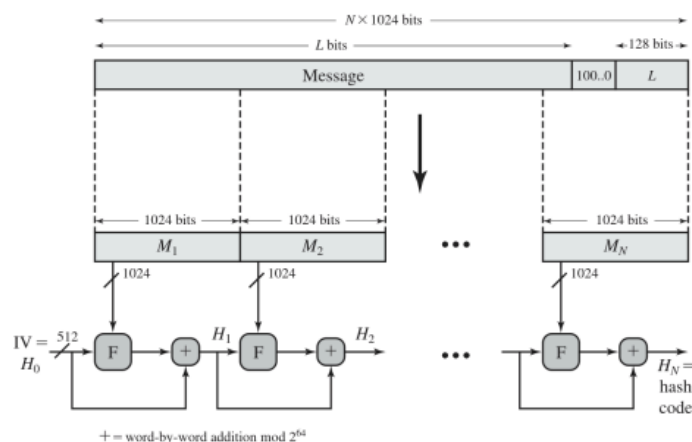
Ogni blocco attraversa un modulo composto da **80 round** (cicli):

- **Input del Round:** Ogni round t utilizza una *word* _{t} (calcolata espandendo il blocco corrente) e una costante crittografica K_t (entrambi a 64 bit).
- **Operazioni:** Vengono eseguite combinazioni di funzioni booleane standard (AND, OR, NOT, XOR), uniche per ogni fase, sommate a spostamenti circolari (Shift/Rotazioni).
- **Aggiornamento:** L'output finale di questi 80 round viene sommato al valore che il buffer aveva *prima* dell'inizio degli 80 round, utilizzando l'addizione modulo 2^{64} ($\text{mod } 2^{64}$).

Output Finale

L'algoritmo ripete lo Step 4 per ogni singolo blocco da 1024 bit del messaggio.

Dopo aver elaborato l'ultimo blocco (N -esimo), i valori contenuti nei registri del buffer a 512 bit rappresentano il **Digest Finale** del messaggio.



Proprietà Fondamentale

L'algoritmo SHA-512 ha la proprietà per cui **ogni bit dell'hash è una funzione di ogni bit dell'input**.

Sebbene SHA-2 sembri inattaccabile, si basa sulle stesse operazioni dei suoi predecessori. Di conseguenza è stata indetta una competizione per la produzione di una funzione hash di nuova generazione, denominata **SHA-3**.

I requisiti di SHA-3 sono:

- Supportare lunghezze del valore hash di 224, 256, 384 e 512 bit, così da consentire la **sostituzione diretta** da SHA-2 a SHA-3.
- Mantenere la **natura online** di SHA-2, elaborando blocchi da 512 o 1024 bit alla volta.

HMAC (Hash-based Message Authentication Code)

Concetto Principale

Negli ultimi anni, l'autenticazione dei messaggi si è spostata verso l'uso di funzioni di hash crittografiche (MAC derivati da hash). I motivi principali sono l'**elevata velocità di esecuzione** (rispetto agli algoritmi simmetrici/asimmetrici tradizionali) e la **vasta disponibilità di librerie** software ottimizzate.

Il Problema dell'Hash Puro

Una normale funzione di hash (come SHA-1 o SHA-256) non è stata progettata per funzionare come MAC. Il motivo è semplice: un hash puro garantisce l'integrità del dato, ma **non utilizza una chiave segreta**. Di conseguenza, chiunque può ricalcolare l'hash di un messaggio modificato. L'**HMAC** nasce proprio per risolvere questo problema, iniettando in modo matematicamente sicuro una chiave segreta all'interno dell'elaborazione di un normale algoritmo di hash.

Obiettivi di Progettazione (L'approccio "Scatola Nera")

L'architettura dell'HMAC è stata pensata per essere modulare e agnostica rispetto all'algoritmo sottostante:

- **Nessuna modifica interna:** Utilizzare le funzioni di hash esistenti senza doverne alterare il codice o la struttura.
- **Sostituibilità (Black-Box):** Trattare la funzione di hash come un modulo intercambiabile. Se un algoritmo diventa obsoleto (es. SHA-1), basta "scollegarlo" e inserire quello nuovo (es. SHA-256) senza riscrivere la logica dell'HMAC.
- **Prestazioni:** Preservare la velocità nativa dell'algoritmo di hash originale.
- **Sicurezza dimostrabile:** Poter dimostrare matematicamente la robustezza dell'HMAC in relazione alla robustezza della funzione di hash scelta.

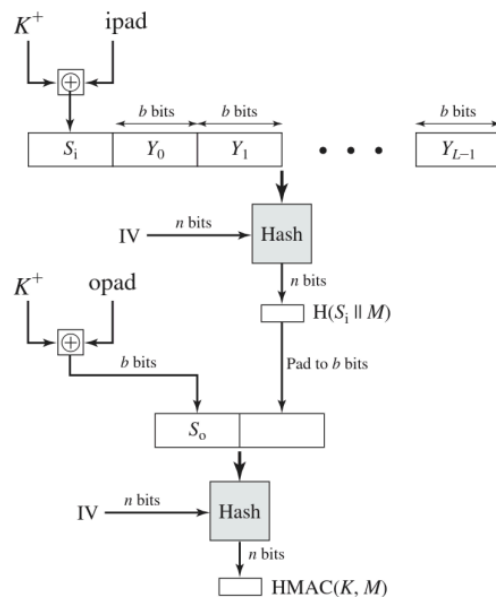
La Formula Matematica dell'HMAC

Tutto il funzionamento dell'HMAC si riassume in questa singola formula:

$$HMAC(K, M) = H((K' \text{ opad}) \ H((K' \text{ ipad}) \ M))$$

Legenda dei parametri:

- H : La funzione di hash scelta (es. SHA-256).
- K : La chiave segreta condivisa.
- M : Il messaggio in chiaro da autenticare.
- K' : La chiave K adattata in lunghezza per essere esattamente pari alla dimensione del blocco di input dell'hash (es. 512 bit). Se K è troppo corta, viene aggiunto un **padding di zeri a destra**. Se è troppo lunga, viene prima "hoshata" ($H(K)$) per accorciarla.
- $ipad$ (Inner pad): Una stringa fissa creata ripetendo il byte `0x36`.
- $opad$ (Outer pad): Una stringa fissa creata ripetendo il byte `0x5C`.
- $\oplus$: Operazione logica XOR bit-a-bit.
- $\parallel$: Concatenazione dei dati.



L'Algoritmo Passo-Passo

Traducendo la formula in operazioni sequenziali, l'HMAC esegue queste fasi:

① Fase 1: Creazione delle Chiavi Mascherate

Attraverso lo XOR con le due costanti ($ipad$ e $opad$), da una singola chiave K' vengono generate in modo pseudocasuale due chiavi interne distinte:

1. S_i (**Stato Interno**): Si calcola $K' \text{ ipad}$.
2. S_o (**Stato Esterno**): Si calcola $K' \text{ opad}$.

② Fase 2: Il Primo Passaggio di Hash (Inner Hash)

3. Al blocco S_i appena creato viene **concatenato** il messaggio M ($S_i \parallel M$).
4. Questo enorme flusso di dati viene dato in pasto alla funzione di Hash H .

5. Il risultato è un **digest intermedio** (o hash interno).

📌 Fase 3: Il Secondo Passaggio di Hash (Outer Hash)

6. Al blocco S_o viene **concatenato** il digest intermedio calcolato al punto precedente.

7. Questo nuovo blocco viene passato un'ultima volta alla funzione di Hash H .

8. Il risultato finale è l'**HMAC** definitivo.

Sicurezza e Resistenza

La forza dell'HMAC deriva da un teorema fondamentale: **la sicurezza dell'HMAC dipende esclusivamente dalla robustezza della funzione di hash incorporata.**

In termini di crittanalisi, per riuscire a falsificare un HMAC (ovvero generare un MAC valido senza conoscere la chiave K), un attaccante deve riuscire a compiere uno di questi due attacchi sulla funzione di hash sottostante:

1. **Forzare la Funzione di Compressione (Brute Force):** L'attaccante deve riuscire a calcolare l'output della funzione di compressione interna avendo un Vettore di Inizializzazione (IV) segreto. Questo richiede un attacco a forza bruta con uno sforzo computazionale pari a 2^n (dove n è la lunghezza dell'hash).
2. **Ricerca di Collisioni (Birthday Attack):** L'attaccante deve riuscire a trovare due messaggi diversi M e M' che producano lo stesso hash interno $H(M) = H(M')$. Sfruttando il paradosso del compleanno, questo attacco richiede uno sforzo teorico pari a $2^{n/2}$.

Authenticated Encryption

È un approccio che garantisce contemporaneamente **integrità** e **autenticità** di un messaggio.

- Per farlo **non si può usare la modalità ECB**, poiché produce due blocchi cifrati identici a fronte di due blocchi in chiaro identici.
- Si usa allora la modalità **OCB (Offset Codebook Mode)**, che introduce un offset tale che ogni blocco $M[i]$ viene "sporcat" tramite $[i]$ prima della cifratura:

$$C[i] = E_k(M[i] \oplus [i]) \oplus [i]$$

Si utilizza un valore **Nonce** per generare l'offset, che deve essere tassativamente cambiato per ogni messaggio inviato con la stessa chiave.

2. Quando A invia Y_A a B, l'attaccante D intercetta Y_A e trasmette invece Y_{D1} a B, calcolando inoltre la chiave di sessione con A: $K_2 = (Y_A)^{X_{D2}} \pmod{q}$.

Algoritmo a Chiave Pubblica RSA

Descrizione dell'algoritmo

L'RSA è un cifrario a blocchi in cui il testo in chiaro e quello cifrato sono rappresentati da numeri interi compresi tra 0 ed $n - 1$, per un certo valore n .

Per un dato blocco di testo in chiaro M e il corrispondente blocco cifrato C :

- **Cifratura:** $C = M^e \pmod{n}$, Il risultato C è il testo cifrato incomprensibile che viaggia sulla rete.
- **Decifratura:** $M = C^d \pmod{n} = (M^e)^d \pmod{n} = M^{ed} \pmod{n}$

Sia il mittente che il destinatario devono conoscere il valore di n ed e , mentre il valore di d deve rimanere noto esclusivamente al destinatario.

Definiamo quindi le chiavi:

- **Chiave Pubblica (PU):** $\{e, n\}$
- **Chiave Privata (PR):** $\{d, n\}$

Requisiti dell'algoritmo

Perché l'algoritmo funzioni correttamente, devono essere soddisfatti tre requisiti fondamentali:

1. È possibile trovare valori di e, d, n tali per cui $M^{ed} \pmod{n} = M$ per ogni $M < n$.
2. È computazionalmente facile calcolare M^e e C^d per ogni valore di $M < n$.
3. Non è possibile (computazionalmente infattibile) determinare d conoscendo solo e ed n .

I primi due requisiti sono facilmente soddisfatti, mentre il terzo è garantito per valori molto elevati di e ed n .

Riguardo al primo requisito, per far sì che $M^{ed} \pmod{n} = M$, è necessario che e e d siano inversi moltiplicativi modulo (n) , dove (n) è la **Funzione Totiente di Eulero** (ovvero il numero di interi positivi minori di n e coprimi con n).

Abbiamo quindi che:

$$ed \pmod{(n)} = 1 \implies d \pmod{(n)} = e^{-1}$$

Ciò è vero solo se d (e di conseguenza e) è coprimo con (n) , ovvero:

$$\text{D}((n), d) = 1$$

Funzionamento dell'algoritmo (Generazione delle Chiavi)

1. Si inizia selezionando due numeri primi molto grandi, p e q .
2. Si calcola $n = p \cdot q$.
3. Si calcola la funzione totiente $(n) = (p - 1)(q - 1)$.
4. Si seleziona un intero e che sia coprimo con (n) .
5. Infine, si calcola d come l'inverso moltiplicativo di e modulo (n) .

☰ Esempio Pratico: Algoritmo RSA

1. Generazione delle Chiavi

- Selezioniamo due numeri primi: $p = 1$ e $q = 11$.
- Calcoliamo il modulo n :

$$n = p \cdot q = 1 \cdot 11 = 11$$

- Calcoliamo la funzione totiente di Eulero (n) :

$$(n) = (p - 1)(q - 1) = 16 \cdot 10 = 160$$

- Selezioniamo l'esponente pubblico e tale che sia coprimo e minore rispetto a (n) .
Ad esempio, scegliamo $e = 7$.
- Determiniamo l'esponente privato d tale che $(e \cdot d) \bmod 160 = 1$ e che sia $d < 160$.
In questo caso otteniamo $d = 23$ (infatti $7 \cdot 23 = 161$, e $161 \bmod 160 = 1$).

Le Chiavi Generate

- **Chiave Pubblica:** $\{e, n\} = \{7, 18\}$
- **Chiave Privata:** $\{d, n\} = \{23, 18\}$

2. Cifratura e Decifratura

Ipotizziamo di voler inviare il messaggio numerico $M = 88$.

- **Fase di Cifratura:** Utilizziamo la chiave pubblica per calcolare il testo cifrato C .

$$C = M^e \bmod n = 88^7 \bmod 18 = 11$$

Il messaggio cifrato da inviare sulla rete è 11.

- **Fase di Decifratura:** Il destinatario utilizza la propria chiave privata per recuperare il messaggio originale M .

$$M = C^d \bmod n = 11^{23} \bmod 18 = 88$$

Il calcolo restituisce esattamente il messaggio di partenza.

📄 Esempio di trasmissione

Per inviare un messaggio M all'utente A, l'utente B calcolerà il crittogramma $C = M^e \pmod n$ utilizzando la chiave pubblica di A, e lo trasmetterà. L'utente A decifrerà il messaggio usando la propria chiave privata calcolando $M = C^d \pmod n$.

Sicurezza dell' algoritmo RSA

Per attaccare l'algoritmo RSA esistono principalmente quattro approcci:

1. **Brute force:** Consiste nel provare tutte le possibili chiavi private. La difesa contro il brute force consiste nell'usare uno spazio di chiavi molto ampio, sebbene questo rallenti il sistema.
2. **Attacchi matematici:** Esistono diversi approcci, tutti equivalenti in termini di sforzo alla fattorizzazione del prodotto di due numeri primi.
3. **Attacchi temporali (Timing Attacks):** Dipendono dall'analisi del tempo di esecuzione dell'algoritmo di decifratura.
4. **Attacchi con testo cifrato scelto:** Sfruttano le proprietà intrinseche dell'algoritmo RSA.

Attacchi Matematici (Il Problema della Fattorizzazione)

📄 Obiettivo dell'attaccante

La sicurezza dell'RSA si basa sull'ipotesi che sia "facile" moltiplicare due numeri primi enormi (p e q), ma sia computazionalmente "impossibile" fare il processo inverso, ovvero **fattorizzare** il risultato (n) per ritrovare i due numeri di partenza.

Per distruggere matematicamente una chiave RSA, un attaccante ha tre strade teoriche, tutte con lo stesso enorme grado di difficoltà computazionale:

1. **Fattorizzare n nei suoi fattori primi (p e q)**: È la via più diretta. Conoscendo p e q , l'attaccante calcola la funzione totiente (ϕ), e da lì ricava facilmente la chiave privata d .
2. **Determinare (n) direttamente**: Senza passare per p e q . Ottenuto (n), si calcola d .
3. **Determinare d direttamente**: Trovare direttamente l'inverso moltiplicativo.

Regole per la scelta dei numeri primi (p e q)

Per evitare algoritmi di fattorizzazione specializzati che sfruttano casi matematici particolari, i due numeri primi scelti devono rispettare dei vincoli precisi:

- p e q **devono avere lunghezze simili**: Ma non devono essere **troppo** vicini tra loro, altrimenti l'algoritmo di fattorizzazione di Fermat li trova in pochi secondi.
- $(p - 1)$ e $(q - 1)$ **devono contenere un grande fattore primo**: Altrimenti l'algoritmo "Pollard's $p-1$ " può bucare il modulo n molto velocemente.
- $\text{D}(p - 1, q - 1)$ **deve essere molto piccolo**: Per garantire che lo spazio delle chiavi resti il più ampio e imprevedibile possibile.

Limitazione sull'esponente privato d (Attacco di Wiener)

Per decifrare un messaggio si esegue il calcolo $C^d \pmod n$. Se d è un numero gigantesco, il processore ci mette più tempo. Spesso i progettisti sono tentati di scegliere un valore di d molto piccolo per velocizzare le operazioni di decifratura sui server.

⚡ L'Attacco di Wiener

La matematica ha dimostrato che se si sceglie un d troppo piccolo per risparmiare tempo (nello specifico, se $d < n^{1/4}$), l'attaccante può dedurre il valore di d istantaneamente analizzando la chiave pubblica attraverso una tecnica matematica chiamata "frazioni continue". Pertanto, d deve essere sempre un numero molto grande.

3. Attacchi a Tempo (Timing Attacks)

Questo è un attacco di tipo *Side-Channel* (canale laterale). L'attaccante non cerca di rompere la matematica dell'RSA, ma "spia" **il comportamento hardware del computer** che esegue la decifratura.

Come funziona l'attacco?

Per calcolare enormi potenze come $C^d \pmod n$, i processori usano un algoritmo ottimizzato chiamato *Square-and-Multiply* (Eleva al quadrato e moltiplica). Questo algoritmo analizza la chiave privata d un bit alla volta, da sinistra verso destra:

- Se il bit è **0**: il processore esegue solo un'operazione (elevamento al quadrato).

- Se il bit è **1**: il processore esegue **due** operazioni (elevamento al quadrato + una moltiplicazione aggiuntiva).

La seconda moltiplicazione richiede qualche frazione di millisecondo in più. Un attaccante che osserva attentamente i tempi di esecuzione del server può letteralmente "ascoltare" il processore: se ci mette poco, il bit della chiave era **0**; se ci mette un po' di più, il bit era **1**. Così facendo, ricostruisce l'intera chiave privata *d* pezzo per pezzo.

Contromisure:

Per tappare questa falla legata all'hardware, si possono adottare tre strategie:

1. **Tempo costante**: Si programma il software in modo che la CPU perda tempo appositamente quando incontra uno **0**, in modo che l'operazione duri esattamente come se fosse un **1**. (Difetto: rallenta pesantemente le prestazioni generali del server).
2. **Ritardo casuale (Random Delay)**: Si aggiungono pause di durata casuale durante i calcoli per confondere l'attaccante. (Difetto: con un numero sufficiente di tentativi, sofisticati modelli statistici possono filtrare il rumore e ricostruire comunque i tempi reali).
3. **Blinding (Accecamento)**: È la contromisura più efficace e usata oggi. Prima di decifrare il crittogramma, lo si moltiplica per un numero casuale segreto (il *blinding factor*). Il processore esegue quindi i calcoli su un dato "mascherato", rendendo l'analisi dei tempi totalmente inutile per l'attaccante. Al termine, il risultato viene "sbendato" matematicamente rimuovendo il numero casuale, ottenendo il testo in chiaro originale.

Scambio di Chiavi Diffie-Hellman

Lo scopo dell'algoritmo Diffie-Hellman è consentire a due utenti di scambiarsi una chiave segreta in modo sicuro, che potrà poi essere usata per la successiva cifratura simmetrica dei messaggi.

L'efficacia di tale algoritmo si basa sulla difficoltà di calcolare i **logaritmi discreti**, che si definiscono a partire dal concetto di radice primitiva:

Definiamo *radice primitiva* di un numero primo p quel numero a le cui potenze generano tutti gli interi da 1 a $p - 1$ ($a^1 \pmod p, a^2 \pmod p, \dots, a^{p-1} \pmod p$).

L'Algoritmo Matematico Passo-Passo

Siano Alice e Bob i due interlocutori che vogliono stabilire una chiave segreta K .

1. Condivisione dei parametri pubblici

Alice e Bob concordano (in chiaro) due numeri:

- p : un numero primo molto grande.
- g : un numero intero (chiamato *generatore* o *radice primitiva* di p) tale che $g < p$.
Questi due numeri viaggiano in chiaro sulla rete e sono noti agli attaccanti.

2. Generazione delle chiavi private e pubbliche

- **Alice** sceglie un numero intero casuale segreto a (dove $a < p$).
Calcola la sua chiave pubblica A :

$$A = g^a \pmod p$$

- **Bob** sceglie un numero intero casuale segreto b (dove $b < p$).
Calcola la sua chiave pubblica B :

$$B = g^b \pmod{p}$$

3. Lo Scambio

Alice invia A a Bob. Bob invia B ad Alice.

(L'attaccante ora conosce p, g, A, B , ma calcolare a o b partendo da questi dati è il problema del logaritmo discreto, attualmente irrisolvibile per numeri sufficientemente grandi).

4. Calcolo del Segreto Condiviso (K)

- **Alice** riceve B e calcola la chiave condivisa usando il suo segreto a :

$$K = B^a \pmod{p}$$

- **Bob** riceve A e calcola la chiave condivisa usando il suo segreto b :

$$K = A^b \pmod{p}$$

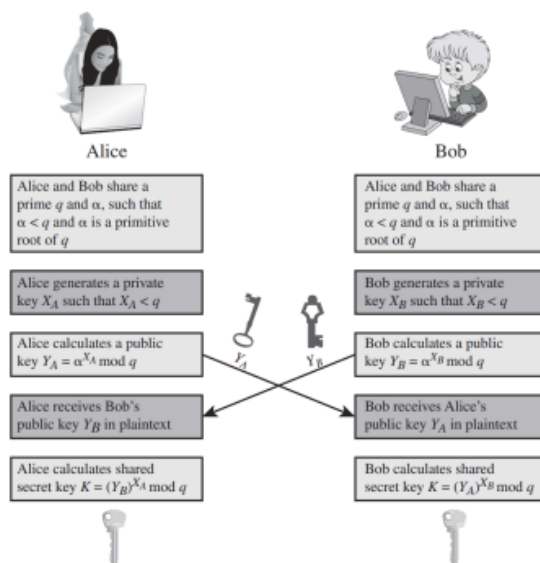
✓ Dimostrazione dell'uguaglianza

Il calcolo di Alice è $K = B^a \pmod{p}$. Poiché sappiamo che $B = g^b \pmod{p}$, possiamo sostituirlo:

$$K = (g^b)^a \pmod{p} = g^{ba} \pmod{p}$$

Per le proprietà commutative degli esponenti, g^{ba} è esattamente uguale a g^{ab} , che è il calcolo che effettua Bob: $(g^a)^b$.

Entrambi arrivano allo stesso identico numero K !



Attacco Man-in-the-Middle (MitM)

L'algoritmo appena illustrato (nella sua forma base) non è sicuro contro un attacco Man-in-the-Middle. Supponiamo che A e B vogliano scambiarsi le chiavi e D sia l'avversario (l'attaccante in mezzo).

1. D può prepararsi all'attacco generando due chiavi private X_{D1} e X_{D2} e calcolando le rispettive chiavi pubbliche Y_{D1} e Y_{D2} .

2. Quando A invia Y_A a B, l'attaccante D intercetta Y_A e trasmette invece Y_{D1} a B, calcolando inoltre la chiave di sessione con A: $K_2 = (Y_A)^{X_{D2}} \pmod{q}$.
3. Bob riceve Y_{D1} e calcola la sua chiave: $K_1 = (Y_{D1})^{X_B} \pmod{q}$, poi trasmette la sua chiave pubblica Y_B .
4. D intercetta anche Y_B e calcola la chiave di sessione con B.
5. A questo punto, D condivide una chiave con A e una chiave diversa con B. D può leggere in chiaro tutti i messaggi inviati tra A e B (decifrandoli con una chiave e ricifrandoli con l'altra), e può anche modificarli a piacimento.

✓ Soluzione

Questa specifica vulnerabilità può essere superata con l'uso di **firme digitali** e **certificati a chiave pubblica** per autenticare l'identità dei partecipanti allo scambio.

Autenticazione degli Utenti

La base della sicurezza informatica è l'accesso controllato: garantire che **qualcuno sia autorizzato a compiere una determinata azione su qualcosa**. Affinché il controllo degli accessi funzioni, dobbiamo essere certi di **chi** sia quel qualcuno. Per farlo ci sono due fasi:

1. **Identificazione**: Affermare chi è una persona.
2. **Autenticazione**: Dimostrare che l'identità affermata sia corretta.

Le identità (es. username) sono genericamente pubbliche; ciò che distingue gli impostori dagli utenti legittimi è la prova fornita tramite l'autenticazione, che invece è un'informazione strettamente protetta.

Utilizzo delle Password

L'utilizzo delle password è un metodo di autenticazione piuttosto semplice: un utente inserisce un dato identificativo (che può essere pubblico) e la password si occupa di fungere da prova per l'autenticazione.

Sorgono però diverse difficoltà operative:

- **Utilizzo**: Fornire una password diversa per ogni accesso a sistemi differenti può essere scomodo.
- **Divulgazione**: Se un utente divulga una password a una persona non autorizzata, l'oggetto protetto diventa accessibile. Se si costringe l'utente a cambiare password, bisogna inoltre informare tutti gli altri utenti legittimi che condividono quell'accesso.
- **Revoca**: Per revocare l'accesso di un utente a un sistema condiviso, bisogna cambiare la password per tutti, creando gli stessi problemi della divulgazione.
- **Perdita**: Potrebbe essere impossibile recuperare una password persa, richiedendo l'assegnazione di una nuova credenziale.

Attacchi e Protezione delle Password

Un attaccante potrebbe tentare di accedere a un account provando a indovinare la password seguendo (in ordine di difficoltà crescente) questi 12 passaggi:

1. Nessuna password.

2. Password uguale all'ID utente.
3. Password derivata dall'ID utente o dal nome.
4. Password presente in un elenco di parole o pattern comuni (es. "password", "qwerty").
5. Password contenuta in un breve dizionario universitario/aziendale.
6. Password contenuta in un elenco completo di parole inglesi.
7. Password contenuta in comuni dizionari non in lingua inglese.
8. Parola del breve dizionario con aggiunta di maiuscole o sostituzioni.
9. Parola del dizionario completo con maiuscole o sostituzioni.
10. Parola di dizionari stranieri con maiuscole o sostituzioni.
11. Ottenuta tramite Bruteforce, provando combinazioni di caratteri alfabetici.
12. Ottenuta tramite Bruteforce totale (tutto il set di caratteri).

Con l'ultimo step ogni password può teoricamente essere indovinata, la sua sicurezza viene quindi determinata dal tempo e dal *numero di tentativi necessari* per trovarla.

Attacchi a Dizionario e Deduzioni

Diversi siti pubblicano dizionari di frasi, nomi di personaggi di fantasia, luoghi, ecc. Un amministratore può scansionare il proprio sistema alla ricerca di password comunemente utilizzate per vietarle, ma la stessa operazione può essere fatta da un malintenzionato.

- **Dedurre le password più probabili:** Le persone in genere scelgono password personali per ricordarle più facilmente. Spesso questo riduce il numero di password plausibili per una determinata persona a una o due dozzine, richiedendo meno di un secondo per essere testate. Secondo uno studio, il 30% delle password ha meno di 7 caratteri, il 50% usa nomi, slang o parole comuni, e un'enorme fetta include pattern banali (12345, 123456, password).
- **Indovinare le password:** Poiché le scelte umane sono solitamente corte e facili da pronunciare, i malintenzionati pianificano i tentativi testando le parole in ordine di lunghezza o esplorando per prime le parole contenute in grandi dizionari.

Superare l'Occultamento (Hash e Rainbow Tables)

Invece di tentare un accesso diretto (online), è spesso più facile per un attaccante rubare la tabella delle password del sistema operativo. Per ragioni di sicurezza, le password in queste tabelle non vengono salvate in chiaro, ma in forma "occultata" tramite una funzione crittografica di hash (unidirezionale). Quando l'utente tenta di autenticarsi, l'OS applica alla password inserita la stessa funzione di occultamento e confronta l'output con il valore memorizzato.

Il vincolo fondamentale è che il processo sia **unidirezionale**, rendendo impossibile invertire matematicamente l'hash per ottenere la password originale.

Tuttavia, se il malintenzionato ottiene la tabella delle password offuscate e conosce l'algoritmo utilizzato, può creare una **Rainbow Table**: una tabella pre-calcolata che associa le password in chiaro più comuni alla loro rispettiva versione offuscata, cercando poi delle corrispondenze dirette (matching).

Inoltre, in un sistema del genere, se due utenti utilizzano la stessa password, avranno anche lo stesso identico hash nel database, facilitando il lavoro dell'attaccante.

✓ Difesa: Il Salt

Per contrastare la minaccia delle Rainbow Tables, i sistemi aggiungono un elemento extra al processo di occultamento chiamato **Salt**.

Il salt è un valore (spesso casuale o legato all'account, come la data di creazione) che viene concatenato alla password prima di calcolarne l'hash.

In questo modo, l'hash di "Password+Salt1" sarà completamente diverso dall'hash di "Password+Salt2". Ciò impedisce a due utenti con la stessa password di avere lo stesso hash nel database e rende inefficaci le Rainbow Tables pre-calcolate.

Attacco Esaustivo (Bruteforce)

In un attacco esaustivo, l'attaccante prova sistematicamente tutte le combinazioni possibili. Per mitigare questi attacchi, l'unica soluzione è forzare l'uso di password molto più complesse o abbinare la password ad altre informazioni (es. domande di sicurezza, controlli contestuali).

Autenticazione tramite Dati Biometrici

I dati biometrici sono proprietà biologiche basate su caratteristiche fisiche del corpo umano. Attualmente i tratti riconoscibili utilizzati includono:

- Impronta digitale
- Geometria della mano
- Retina e Iride
- Voce
- Scrittura a mano (dinamica della firma)
- Caratteristiche di digitazione (Keystroke dynamics)
- Vasi sanguigni nel dito o polso
- Tratti del viso

L'autenticazione biometrica presenta un grande vantaggio rispetto alle password: i dati biometrici non possono essere (facilmente) persi, rubati o condivisi, e sono sempre disponibili con l'utente. Alcune caratteristiche possono essere falsificate, ma il grado di difficoltà tecnica per farlo è generalmente molto alto.

Funzionamento:

L'autenticazione richiede la creazione di un *pattern* o *template* (fase di enrollment) che rappresenti la misurazione base della caratteristica. Quando un utente tenta di autenticarsi, viene effettuato un nuovo set di misurazioni che viene confrontato con il template salvato. Se la corrispondenza è "sufficientemente vicina" (es. il 90% dei valori corrisponde, oppure rientra in una tolleranza del 5%), l'autenticazione ha successo. Fare misurazioni così precise richiede tempo e potenza di calcolo, fattori determinanti per decidere l'idoneità del sistema in un particolare ambiente.

Accuratezza nell'Autenticazione Biometrica

L'autenticazione biometrica si basa su un sistema di screening che possiamo definire come un *Test Dicotomico* (il sistema deve decidere: c'è matching o non c'è matching). Vogliamo evitare due tipi di errori fondamentali: **Falsi Positivi (FP)** (un impostore viene accettato) e **Falsi Negativi (FN)** (l'utente legittimo viene respinto).

Per valutare l'accuratezza del sistema si usano 4 parametri statistici principali:

- **Sensibilità (True Positive Rate):** Misura la capacità di identificare correttamente i positivi reali.

$$\text{Sensibilità} = \frac{TP}{TP + FN}$$

- **Specificità (True Negative Rate):** Misura la capacità di identificare correttamente i negativi reali.

$$\text{Specificità} = \frac{TN}{TN + FP}$$

- **Accuratezza Totale:** Misura la capacità complessiva di dare risposte corrette.

$$\text{accuratea} = \frac{TP + TN}{TP + FP + FN + TN}$$

- **Prevalenza:** Misura quanto è comune una determinata condizione o situazione all'interno del campione analizzato.

$$\text{revalena} = \frac{TP + FN}{TP + FP + FN + TN}$$

Valore Predittivo e Curva ROC

Sensibilità e specificità sono statisticamente correlate in un sistema biometrico: regolando la soglia di tolleranza, quando una aumenta, inevitabilmente l'altra diminuisce.

Si può calcolare il **Valore Predittivo** di un test, che esprime la probabilità che il risultato finale sia corretto:

- **Valore Predittivo Positivo:** $\frac{TP}{TP + FP}$
- **Valore Predittivo Negativo:** $\frac{TN}{TN + FN}$

Una **Curva ROC (Receiver Operating Characteristic)** è una rappresentazione grafica fondamentale che pone la Sensibilità sull'asse Y e $(1 - \text{Specificità})$ sull'asse X, utile per calibrare il sistema.

Problemi legati all'uso della Biometria

La biometria è purtroppo associata a diversi problemi pratici e di sicurezza:

- **Invasività:** Alcune persone la trovano psicologicamente o fisicamente invasiva.
- **Single Point of Failure:** I lettori o i sistemi di confronto centralizzati possono diventare un punto critico di vulnerabilità.
- Le soglie di tolleranza introducono sempre un margine di approssimazione (le letture esatte al 100% sono rare).
- **Lettture errate:** Si verificano a causa di fattori esterni o usura del sensore.
- La velocità con cui deve essere effettuato un riconoscimento ne limita l'uso in contesti ad alto traffico.
- **Falsificazioni:** Le tecniche di spoofing (creazione di impronte finte, maschere, registrazioni vocali) sono possibili.

- **Accuratezza limitata:** Il fatto che le impronte siano uniche non ha una base scientifica assoluta e inconfutabile; inoltre, i sistemi analizzano spesso solo dei *template parziali* (minuzie), non l'intera impronta. I dati biometrici possono poi variare per fattori fisici (ferite, sporcizia) o psicologici (lo stress altera il tono della voce).

Tutto questo rende la biometria, allo stato attuale, più un "gioco di probabilità" che una certezza matematica. Per questo motivo è considerata affidabile per l'autenticazione (verifica 1 a 1), ma molto meno per l'identificazione di massa (ricerca 1 a N).

Utilizzo di Token e Sistemi Gestionali

Token Attivi e Passivi

- **Token Passivi:** Non fanno nulla attivamente (es. una classica chiave fisica). Possono avere una certa variabilità o interagire con l'ambiente circostante (es. carte con banda magnetica o smart card base).
- **Token Attivi:** Sono dispositivi elettronici che elaborano dati (es. chiavette generatrici di OTP).

Token Statici e Dinamici

- Il valore di un **Token Statico** rimane fisso nel tempo. Sono particolarmente utili per l'autenticazione "in loco" (es. strisciare un badge o mostrare una Carta d'Identità). Ci interessa però anche l'autenticazione remota (online), dove l'uso di token statici espone al rischio di falsificazioni o di **Skimming** (l'uso di un dispositivo hardware/software per copiare furtivamente i dati di autenticazione).
- Per prevenire lo skimming, si utilizzano i **Token Dinamici**, il cui valore cambia nel tempo. Si tratta essenzialmente di dispositivi (hardware o software) che generano un valore imprevedibile valido solo per un singolo accesso, rendendo inutile per un attaccante la clonazione di un codice già usato.

Federated Identity Management (FIM) e Single Sign-On (SSO)

- **FIM (Federated Identity Management):** È l'unione di sistemi di identificazione e autenticazione separati all'interno di un unico schema fiduciario. I diversi sistemi (che possono appartenere ad aziende diverse) condividono un database centralizzato e usano un unico modulo di gestione dell'identità.
- **SSO (Single Sign-On):** Prevede che i sistemi applicativi siano separati, ma l'utente si autentichi una sola volta al primo "Sign-On". Le credenziali (o i token generati) vengono memorizzati, e le autenticazioni verso i servizi successivi avvengono in automatico senza richiedere un nuovo inserimento.

Autenticazione a Più Fattori (MFA)

Possiamo compensare le limitazioni e le vulnerabilità di una singola forma di autenticazione combinandola con un'altra di natura diversa (Qualcosa che sai, Qualcosa che hai, Qualcosa che sei).

Un classico esempio è la **2FA (Two-Factor Authentication)** tramite una password abbinata a un codice temporaneo (OTP) inviato via SMS o Email.

Ciò rende l'autenticazione drasticamente più robusta contro gli attacchi, ma inevitabilmente anche più complessa da gestire lato infrastruttura e potenzialmente più frustrante da utilizzare per l'utente finale. Inoltre, aggiungere più fattori non migliora la sicurezza se l'infrastruttura di base di quei fattori (es. la rete cellulare per gli SMS) è compromessa.

Problemi di Sicurezza dell'Autenticazione (Vettori d'Attacco)

I sistemi di autenticazione sono soggetti a diverse tipologie di attacchi di rete e di sistema:

- **Attacchi Client:** L'avversario tenta di ottenere l'autenticazione dell'utente senza avere accesso all'Host remoto o al percorso di comunicazione intermedio (es. tentando di indovinare o forzare la password direttamente dal form di login).
- **Attacchi all'Host:** Sono diretti ai file di sistema sull'Host dove sono fisicamente memorizzate le password (o i relativi hash), i token pre-condivisi o i modelli biometrici.
- **Intercettazione (Eavesdropping):** È il tentativo di apprendere la password osservando l'utente o il traffico di rete in chiaro, oppure trovandone una copia scritta incustodita, o tramite software di *Keylogging*.
- **Trojan Horse:** Un'applicazione, un sito o un dispositivo fittizio si maschera da entità autentica per ingannare l'utente e acquisirne password o dati d'accesso (es. Phishing).
- **Replay Attack:** Un avversario intercetta e cattura una risposta di autenticazione valida dell'utente inviata in precedenza, per poi "ripeterla" (inviarla nuovamente) al server per ottenere l'accesso illegittimo.
- **Denial-of-Service (DoS):** Tenta di disabilitare l'intero servizio di autenticazione inondandolo con innumerevoli tentativi di login o richieste malformate. Un attacco più selettivo può mirare a negare il servizio a un singolo utente specifico (es. causando volutamente il blocco dell'account per troppi tentativi errati).

Controllo degli Accessi

⚡ Il paradigma

Un soggetto è autorizzato ad accedere a un oggetto in una particolare modalità e sono consentiti solo accessi autorizzati.

- **Soggetti:** Sono utenti umani, spesso tramite programmi che li rappresentano.
- **Oggetti:** Sono elementi su cui è possibile eseguire un'azione, ad esempio file, ma anche i soggetti stessi (il Sistema Operativo può agire su un utente o un programma).
- **Modalità d'accesso:** Sono tutte le azioni controllabili dei soggetti sugli oggetti.

Access Policies

Il controllo degli accessi è facilmente implementabile, ma prima un'organizzazione deve stabilire delle *regole per determinare quali permessi assegnare ai soggetti*.

Implementazione efficace delle politiche

La protezione degli oggetti implica diversi obiettivi complementari:

- **Verificare periodicamente i diritti d'accesso:** Lavori e situazioni cambiano, e i detentori di privilegi potrebbero volerli mantenere anche quando non più necessari.
- **Applicare il principio del minimo privilegio:** Un soggetto dovrebbe avere accesso al numero minimo di oggetti necessari per svolgere un determinato compito.
- **Verificare l'utilizzo accettabile:** Bisogna controllare che l'attività avvenuta su un oggetto sia appropriata.

Privilegio Limitato

È strettamente legato al minimo privilegio. È l'atto di limitare i soggetti in modo che qualsiasi potenziale danno sia contenuto. Il processo di analisi degli utenti e di determinazione dei privilegi di cui necessitano è fondamentale per autorizzare l'accesso entro tali limiti.

Monitoraggio

A volte gli amministratori devono rivedere la politica di accesso per determinare se funziona come dovrebbe. Ciò viene fatto rispondendo ad almeno due domande:

1. A quali oggetti e sistemi può accedere un determinato utente?
2. Per ogni oggetto e sistema, chi ha accesso?

Granularità

È la precisione o la specificità del controllo degli accessi, ed è una scala che va da un estremo all'altro.

- A un estremo si controlla l'accesso a ogni singolo bit o byte (il che è spesso eccessivo e complesso).
- All'altro estremo viene dato accesso completo al computer.

Pertanto, è necessario trovare un punto intermedio ragionevole. Inoltre, bisogna tenere conto anche degli altri dispositivi e sistemi collegati. In genere, l'unità più piccola a cui viene controllato l'accesso è un file, un programma o uno spazio dati.

Access Log

I sistemi registrano quali accessi sono stati consentiti, creando il "registro di controllo". Questo log può essere utile per pianificare aggiornamenti, identificare le cause di guasti, monitorare l'utilizzo improprio degli oggetti o per ragioni di sicurezza.

Implementazione del Controllo degli Accessi

Il controllo degli accessi viene spesso eseguito dal Sistema Operativo (SO). Tuttavia, l'attuale architettura hardware non sempre supporta il SO nell'implementazione di un ottimo controllo degli accessi (es. il SO non ha accesso all'interno dei file e non può eseguire un controllo riga per riga) e, di conseguenza, questo si affida a dispositivi e gestori più specifici (es. un DBMS per un database). Ciò però porta il SO ad avere difficoltà nel controllare il gestore, e il gestore ad avere un controllo limitato sugli utenti. Pertanto, l'architettura hardware limita il SO.

Monitor di Riferimento

Il controllo degli accessi dipende da una combinazione di Hardware e Software che è:

- *Sempre disponibile* in modo da poter validare ogni tentativo di accesso; **Ogni singola volta** che un programma cerca di leggere un file o accedere a una risorsa, la richiesta deve passare per il Monitor.
- *Immune da manomissioni*, i programmi "normali" non devono avere il potere di disattivare o modificare il codice del Monitor;
- *Sicuramente corretta*, il codice che gestisce questo controllo deve essere **piccolo, semplice e lineare**.

Questa struttura è chiamata **Monitor di riferimento** è un **modello ideale** usato in informatica per descrivere come *dovrebbe* funzionare un sistema di sicurezza perfetto. Perché sia efficace è necessario considerare mezzi pratici ed efficienti per tradurre le politiche in azioni.

Modelli e Strutture Dati per il Controllo degli Accessi

DAC

Il **DAC (Discretionary Access Control - Controllo Discrezionale degli Accessi)** è uno dei modelli fondamentali per la gestione dei permessi nei sistemi operativi.

- **Proprietà (Ownership)**: Ogni oggetto nel sistema (un file, una cartella, un documento) ha un creatore, che ne diventa automaticamente il "proprietario".
- **Potere al Proprietario**: Il proprietario ha il diritto esclusivo di stabilire, modificare o revocare le politiche di accesso per quell'oggetto. Può decidere liberamente di dare i permessi di lettura, scrittura o esecuzione ad altri utenti o gruppi.

Directory di Controllo degli Accessi

È un approccio basilare per la gestione della sicurezza. In questo modello, a **ogni utente viene associata una directory** (che funge da lista delle autorizzazioni) contenente tutti gli oggetti (es. file) a cui ha il permesso di accedere.

- La directory è rigorosamente di **sola lettura** per l'utente, per evitare che si auto-assegni privilegi.
- Ogni file ha un **proprietario**, che è l'unico a detenere i poteri di controllo e a decidere chi può accedere.

Vantaggi

- **Semplicità:** È un meccanismo molto intuitivo, diretto e facile da implementare a livello di base.

Svantaggi e Problematiche Questo approccio non scala bene nei sistemi complessi e presenta quattro criticità principali:

1. **Problemi di scalabilità (Elenchi enormi):** In un sistema in cui si condividono molti oggetti, le directory dei singoli utenti diventano lunghissime, sprestando memoria e rallentando i controlli.
2. **Inefficienza nella revoca dei permessi:** Se l'utente *A* (proprietario) decide di bloccare l'accesso al suo file *F* a tutto il sistema, il Sistema Operativo è costretto a scansionare le directory di *tutti* gli utenti per trovare ed eliminare quel permesso.
 - **Propagazione incontrollata:** L'utente *B* potrebbe aver delegato l'accesso a *C* senza che *A* (il proprietario originale) ne fosse a conoscenza, rendendo la revoca totale ancora più difficile.
3. **Conflitti nello spazio dei nomi (Name collision):** Se due utenti *A* e *B* hanno ciascuno un file chiamato *F* e vogliono dare l'accesso a un utente *S*, si crea un'ambiguità: la directory di *S* non può ospitare due file con lo stesso identico nome.
4. **Overhead gestionale:** Per risolvere il problema dei nomi omonimi, si è costretti a rinominare i file (ad esempio in *A:F* e *B:F*). Questo workaround introduce una notevole complessità gestionale per il Sistema Operativo e risulta scomodo lato utente.

Matrice di Controllo degli Accessi

Un'alternativa più strutturata alla directory è la rappresentazione dei permessi tramite una **matrice bidimensionale**:

- **Righe:** Rappresentano i **Soggetti** (utenti o programmi in esecuzione).
- **Colonne:** Rappresentano gli **Oggetti** (file, risorse, hardware).
- **Celle (Intersezioni):** Contengono i **diritti di accesso** (es. lettura, scrittura, esecuzione) che quel determinato soggetto possiede su quel determinato oggetto.

Per ottimizzare lo spazio, la matrice viene spesso "compressa" e implementata come un **elenco globale di triple**: `<Soggetto, Diritti Oggetto,>`

In questo modo, il sistema memorizza *solo* le celle che contengono effettivamente dei permessi, ignorando tutte le combinazioni vuote.

		objects		
subjects		File A	Printer	System Clock
	User W	Read Write Own	Write	Read
	Admin		Write Control	Control

Svantaggio: Inefficienza nella Ricerca Sebbene l'elenco di triple risolva il problema dello spreco di memoria e possa essere ordinato, introduce un grave problema di performance:

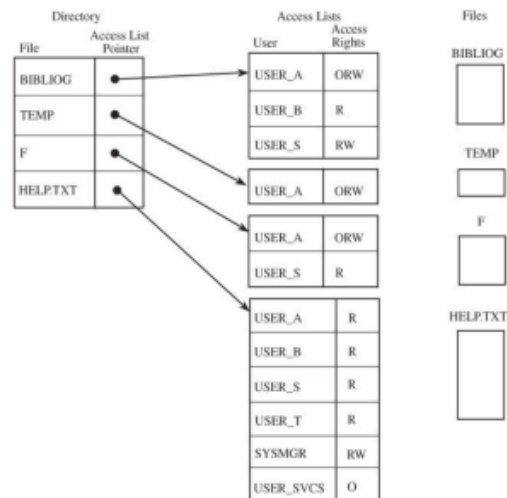
- **Ricerca lenta:** Ogni singola volta che un soggetto richiede l'accesso a un oggetto, il Sistema Operativo deve scorrere o interrogare questa grande lista globale per trovare la tripla corrispondente. Questo processo rende la verifica dei permessi computazionalmente inefficiente.

Lista di Controllo degli Accessi (ACL - Access Control List)

Approccio basato sulle Colonne (Orientato agli Oggetti)

L'ACL associa a **ogni singolo Oggetto** (es. un file) una lista che specifica esattamente chi può accedervi e cosa può farci.

- **Struttura:** L'elenco è formato da coppie `<Soggetto, Diritti>`.
- **Supporto per Gruppi e Ruoli:** Il Soggetto non deve essere per forza un singolo utente, ma può rappresentare gruppi o compartimenti. Ad esempio, una regola come `<Gruppo_Ammistratori, R>` indica che tutti i membri di quel gruppo hanno i diritti di lettura.
- **Logica First-Match:** Le regole all'interno dell'ACL sono ordinate rigorosamente **dal più specifico al più generico**. Quando c'è un tentativo di accesso, il sistema scorre la lista dall'alto verso il basso e applica la *prima regola* che trova corrispondente.
- **Regole di Default:** Sfruttando l'ordinamento e la logica first-match, è prassi inserire in fondo alla lista una regola "jolly" (come `*_*_*`) per definire i diritti di accesso di default per chiunque non rientri nelle regole precedenti.



Elenco dei Privilegi (Capability List)

Approccio basato sulle Righe (Orientato ai Soggetti)

Al contrario dell'ACL, la Capability List associa la lista dei permessi a **ogni singolo Soggetto** (utente o processo). Funziona come un "portachiavi" personale.

- **Struttura:** Per un dato utente, la lista elenca tutti gli Oggetti a cui ha accesso e i relativi diritti (es. `<Oggetto, Diritti>`).
- **Vantaggio (Gestione e Revoca Utente):** Questo modello rende estremamente efficiente la gestione lato utente. Se un dipendente si dimette e deve essere rimosso dal sistema, basta eliminare la sua Capability List. In questo modo si revocano istantaneamente tutti i suoi accessi, operazione che con l'ACL richiederebbe di scansionare ogni singolo file del sistema.
- *(Nota da ricordare per contrasto: se da un lato la Capability List rende facile revocare tutti i permessi a un utente, rende invece difficile revocare a tutti gli utenti l'accesso a un singolo file).*

Capacità (Capabilities)

Una capacità è un **token digitale non falsificabile** che funge da "chiave d'accesso". Conferisce a chi lo possiede determinati diritti su uno specifico oggetto. Teoricamente, può essere visto come una tripla `<Soggetto, Diritti, Oggetto>`, ma all'atto pratico è un pass che il soggetto esibisce per accedere all'oggetto.

Come si garantisce che non sia falsificabile?

Se l'utente potesse crearsi i token da solo, il sistema sarebbe compromesso. Per evitare la falsificazione, il Sistema Operativo adotta una di queste due strategie:

- **Gestione Indiretta (Lato OS):** Il token non viene mai consegnato fisicamente in mano all'utente. Il Sistema Operativo lo custodisce nel proprio spazio protetto e fornisce all'utente solo un "riferimento" (come un numero di targa) per usarlo.
- **Cifratura (Crittografia):** Il token viene consegnato all'utente, ma è criptato o firmato digitalmente con una chiave segreta che solo il Sistema Operativo conosce. Se il token include anche l'identità del proprietario legittimo, un ladro non potrebbe usarlo anche se riuscisse a rubarlo.

Trasferimento e Domini di Esecuzione

- **Trasferibilità (Delega):** Se i diritti della capacità lo consentono, un utente *A* può "prestare" o trasferire una copia del suo token a un utente *B*.
- **Domini di esecuzione:** Quando un processo è in esecuzione, opera all'interno di un "dominio", ovvero la bolla che contiene tutti gli oggetti a cui ha accesso. Se il processo principale chiama una *sottoprocedura* (es. una funzione esterna), può passarle temporaneamente l'accesso ad alcuni oggetti cedendole le relative capacità.

Come nascono le Capacità?

Le capacità spesso non esistono nel vuoto, ma sono supportate "dietro le quinte" da un'ACL o da una matrice.

- **Funzionamento:** La prima volta che un soggetto cerca di aprire un file, il Sistema Operativo controlla l'ACL. Se l'accesso è consentito, l'OS "stampa" una capacità e la consegna al processo. Per tutti gli accessi successivi a quel file, il processo userà direttamente la capacità, rendendo le operazioni molto più veloci.

Memorizzazione Protetta (Protezione in Memoria)

Se le capacità risiedono nella memoria del computer, bisogna impedire che un programma malevolo vada a sovrascriverle o modificarle. I metodi principali sono:

- **Segmenti Nascosti:** Vengono salvate in aree di memoria (segmenti) che non sono mappate nelle tabelle del processo utente, rendendole letteralmente invisibili.
- **Registri Base/Limite:** Si racchiudono le capacità in un'area di memoria blindata, i cui confini fisici sono definiti da speciali registri del processore che impediscono l'accesso non autorizzato.
- **Architettura a Tag (Hardware):** Il processore stesso utilizza un "tag" (un bit extra nella memoria) per etichettare i dati. Se un dato è taggato come "capacità", l'hardware bloccherà qualsiasi tentativo del

programma utente di modificarlo come se fosse una normale variabile.

Il problema della Revoca

Il difetto principale dei pass fisici è che è difficile revocarli (se ti do le chiavi di casa mia, per impedirti di entrare dovrei cambiare la serratura).

- **Soluzione:** Si utilizza una **Tabella Globale delle Capacità**. Invece di far puntare il token direttamente al file, il token punta a una riga di questa tabella centrale, la quale a sua volta punta al file. Se l'amministratore vuole revocare l'accesso, gli basta distruggere o invalidare il puntatore nella tabella centrale; da quel momento, tutti i token in circolazione punteranno a un riferimento "morto" e smetteranno di funzionare.

Paradigmi di Controllo degli Accessi Avanzati

Controllo degli Accessi Orientato alle Procedure

Questo approccio applica alla sicurezza un concetto che conosci bene: l'**Incapsulamento** (tipico della programmazione orientata agli oggetti). Invece di dare all'utente l'accesso diretto ai dati grezzi (es. i file di un database), il Sistema Operativo avvolge l'oggetto in una "capsula". L'utente può interagire con i dati *solo* invocando delle procedure specifiche fornite dal sistema (es. metodi come `add()`, `delete()`, `check()`).

- **Vantaggio:** Le procedure fungono da "filtro intelligente". Possono eseguire controlli di validazione e logiche di sicurezza complesse prima di effettuare la modifica, garantendo che i dati non vengano mai corrotti o letti in modo improprio.
- **Information Hiding (Occultamento):** L'utente conosce solo l'interfaccia pubblica (le funzioni che può chiamare), ma il modo in cui i dati sono fisicamente memorizzati e protetti rimane nascosto e inaccessibile.

Controllo degli Accessi Basato sui Ruoli (RBAC)

Il controllo degli accessi basato sui ruoli permette di associare i privilegi ai "ruoli" (o gruppi funzionali) anziché ai singoli utenti, rendendo la gestione della sicurezza molto più semplice e scalabile.

L'RBAC è uno standard industriale e si struttura in 4 livelli incrementali:

RBAC0 (Modello Base)

Definisce le fondamenta del sistema tramite 4 entità:

- **Utente:** L'identità fisica (es. Mario Rossi).
- **Ruolo:** La funzione operativa all'interno del sistema (es. Sviluppatore).
- **Permesso:** L'autorizzazione tecnica a compiere un'azione (es. "Scrittura su database").
- **Sessione:** Il momento in cui l'utente fa il login e "attiva" le autorizzazioni del suo ruolo.

RBAC1 (Gerarchie di Ruoli)

Introduce il concetto di **Ereditarietà**. Un ruolo di livello superiore eredita in automatico tutti i permessi dei ruoli che gli stanno sotto. *(Es. Il ruolo "IT Manager" eredita automaticamente tutti i permessi del ruolo "Tecnico Base", più i propri).*

RBAC2 (Vincoli)

Introduce regole di logica aziendale per evitare abusi (spesso chiamato *Separation of Duties*):

- **Ruoli mutualmente esclusivi**: Un utente non può avere contemporaneamente due ruoli in conflitto. *(Es. non puoi essere sia colui che fa una richiesta di spesa, sia colui che l'approva).*
- **Cardinalità**: Limiti numerici. *(Es. il ruolo "Rettore" può essere assegnato a un solo utente alla volta).*
- **Ruolo prerequisito**: Per avere il ruolo *Y* devi necessariamente possedere già il ruolo *X*.

RBAC3 (Modello Completo) L'unione di tutte le caratteristiche precedenti: Base (0) + Gerarchie (1) + Vincoli (2).

Controllo degli Accessi Basato sugli Attributi (ABAC)

È il modello più flessibile e dinamico in assoluto. Invece di usare liste fisse o ruoli rigidi, l'ABAC calcola i permessi in **tempo reale** valutando una serie di variabili, chiamate *attributi*. Funziona con una logica condizionale: *SE (condizioni soddisfatte) ALLORA (concedi accesso)*

Le decisioni vengono prese incrociando tre categorie di attributi:

- **Attributi del Soggetto**: Chi sei? *(Es. dipartimento di appartenenza, livello di sicurezza, matricola).*
- **Attributi dell'Oggetto**: Cosa stai cercando di toccare? *(Es. livello di segretezza del documento, data di creazione, autore).*
- **Attributi Ambientali (Contesto)**: In che situazione siamo? *(Es. connessione da IP interno, orario di ufficio, livello di minaccia del sistema basso).*

Il problema delle performance: Poiché l'ABAC deve ricalcolare tutte queste variabili a ogni singolo accesso, in passato era considerato troppo "pesante" per i sistemi operativi tradizionali. Oggi, con architetture come il Cloud e i Web Services (dove la latenza di rete copre i tempi di calcolo), l'ABAC è diventato lo standard per i sistemi distribuiti moderni.

Modello di Policy

È l'insieme delle regole logiche (le *policy*, appunto) scritte dagli amministratori di sistema. Il Modello di Policy stabilisce matematicamente **come gli attributi devono essere combinati** per concedere o negare un accesso.

- **Come funziona**: Usa operatori booleani (AND, OR, NOT) per creare condizioni "Se/Allora" (If-Then).
- **Esempio pratico**: Una policy non dice "Mario può leggere il file X". Dirà invece:
 - *SE* (Soggetto.Ruolo == "Studente L-31")
 - *E* (Oggetto.Tipo == "Tesi")
 - *E* (Ambiente.Connessione == "Rete_Ateneo")

- **ALLORA** Accesso = Consentito.

Modello di Architettura

È l'infrastruttura software vera e propria (i vari componenti del sistema operativo o del server) che intercetta la richiesta dell'utente, valuta la policy e applica la decisione in tempo reale. In un'architettura ABAC standard, il lavoro è diviso tra componenti specifici (spesso indicati con i loro acronimi in inglese):

- **Il Punto di Controllo (PEP - Policy Enforcement Point):** È il "casello" all'ingresso. Quando provi ad aprire un file, il PEP blocca la tua richiesta, la impacchetta e la manda al cervello del sistema per chiedere il permesso. Quando riceve la risposta (Sì/No), sblocca la porta o ti dà errore.
- **Il Punto di Decisione (PDP - Policy Decision Point):** È il "cervello". Riceve la richiesta dal PEP, va a leggere le regole (le Policy) e valuta gli attributi per calcolare il risultato finale.
- **Il Punto di Informazione (PIP - Policy Information Point):** È il fornitore di dati. Se il cervello (PDP) sta valutando una regola e si accorge che gli manca un attributo (ad esempio non sa in che dipartimento lavori), interroga il PIP, che va a pescare quel dato dal database degli utenti.

Programmi e Programmazione

Per descrivere i bug nel software c'è una terminologia standard:

- **Error:** È l'errore commesso da un essere umano.
- **Fault:** È lo step, comando o processo errato.
- **Failure:** È la deviazione dal comportamento richiesto (il guasto effettivo).

Errori di Programmazione Non Intenzionali

Le vulnerabilità di un programma possono avere due tipi di implicazioni per la sicurezza:

1. Possono causare problemi di integrità che portano a risultati o azioni dannose.
2. Offrono un'opportunità di sfruttamento da parte di un malintenzionato.

Un difetto nato per cause innocue come un semplice errore di distrazione può trasformarsi in una vulnerabilità se un malintenzionato scopre come sfruttarlo per manipolare il comportamento del programma. Tra le falle più comuni troviamo: *Buffer Overflow*, *Incomplete Mediation*, *TOCTOU*, *Backdoor*, *Race Condition*, ecc...

Buffer Overflow

Un *buffer* è uno spazio di memoria. Un **buffer overflow** si verifica quando un programma supera lo spazio allocato per il buffer. I buffer overflow sono spesso il punto di partenza per sferrare attacchi più dannosi.

Il programma nativo di Windows `dialer.exe` (utility per comporre numeri di telefono) causava un buffer overflow dello stack se si inseriva un numero maggiore di 100 cifre, permettendo così di sovrascrivere l'indirizzo di ritorno.

In memoria, codice e dati sono indistinguibili tra loro e gli attaccanti sfruttano ciò trattando il codice come dati o viceversa.

Danni da un Buffer Overflow

In memoria non esiste una netta separazione tra *User Space* e *System Space* e, di conseguenza, un attaccante può sfruttare un buffer overflow per sovrascrivere il codice del Sistema Operativo (SO), facendogli eseguire istruzioni con privilegi elevati (**Privilege Escalation**).

Un altro tipo di attacco può avvenire tramite lo **Stack**: quando viene chiamata una sotto-funzione, al suo interno vengono memorizzati i dati, l'indirizzo di ritorno e il puntatore allo stack. Di conseguenza, un attaccante può sfruttare un buffer overflow per modificare l'indirizzo o il puntatore, reindirizzando l'esecuzione a un codice scritto da lui.

Implicazioni della Sovrascrittura della Memoria

Supponiamo di avere il seguente codice C:

```
char sample[10];
for(i=0; i<=9; i++)
    sample[i] = 'A';
sample[10] = 'B'; // Errore: l'indice 10 è fuori dai limiti dell'array (0-9)
```

A seconda di cosa si trova in memoria *dopo* l'array, si possono verificare 4 casi a causa dell'inserimento della lettera 'B' fuori limite:

1. La 'B' **sovrascrive una variabile** dello stesso programma: il danno resta confinato al programma stesso.
2. La 'B' **sovrascrive un'istruzione** dello stesso programma: se non è stata ancora eseguita, il programma crasha o ha un comportamento imprevedibile.
3. (e 4) I problemi di sicurezza sono ben più gravi se la 'B' **sovrascrive lo spazio di sistema**, causando crash completi o portando alle vulnerabilità (privilege escalation) citate in precedenza.

Heap e Stack

A differenza dello stack, l'**Heap** serve per i dati dinamici, cioè quelli di cui non si conosce la dimensione esatta al momento della compilazione.

Poiché lo stack cresce verso il basso e l'heap verso l'alto, questi potrebbero scontrarsi, ma il SO li monitora per evitarlo. Tuttavia, il SO non può controllare il comportamento interno del programma, e se l'attaccante calcola con precisione la quantità di dati da inserire in un buffer overflow, potrebbe sovrascrivere il *Program Counter* memorizzato nello Stack Frame.

Contromisure contro il Buffer Overflow

La contromisura più ovvia è "rimanere nei limiti", il che è una responsabilità condivisa tra programmatore, SO, compilatore e hardware.

- **Controlli di programmazione:** Le tecniche per individuare le situazioni di overflow prima che diventino problemi includono la revisione del codice da parte di terzi e i test indipendenti.
- **Caratteristiche del linguaggio:** Alcuni linguaggi di programmazione (come *Java*, *Perl*, *Python*) generano codice per controllare i limiti prima di memorizzare i dati. I linguaggi "non controllati" (*C*, *C++*, *Assembly*) consentono un accesso ai programmi pressoché illimitato; di conseguenza, gli sviluppatori di compilatori hanno creato estensioni e librerie che generano codice per tenere sotto controllo i programmatori.
- **Analizzatori di codice:** Un analizzatore statico di codice analizza il codice sorgente per rilevare condizioni non sicure. Tuttavia, non sono perfetti.

✓ Ostacoli difensivi (Il Canarino)

L'obiettivo è proteggere lo stack e possiamo farlo avvolgendo ogni frame dello stack in uno strato protettivo, chiamato "**Canarino**".

Poiché l'attaccante spesso non conosce l'indirizzo esatto in cui è memorizzato il program counter, la memoria viene scritta in modo sequenziale (lineare), per raggiungere l'Indirizzo di Ritorno l'attaccante "inonda" il buffer, straripando e sovrascrivendo tutto ciò che trova sul suo cammino.

deciso di inserire un "ostacolo" in mezzo a questo cammino. Quando una funzione viene chiamata, il sistema genera un numero segreto e casuale (il Canarino) e lo posiziona strategicamente nello stack **esattamente tra il buffer (le variabili locali) e l'Indirizzo di Ritorno**.

Quando la funzione termina, prima di saltare all'indirizzo di ritorno, il sistema controlla che il valore del canarino sia intatto. Se è stato modificato, rileva l'overflow e blocca l'esecuzione.

Mediazione Incompleta

La **mediazione incompleta** si verifica quando *un sistema si fida di informazioni fornite da una fonte potenzialmente ostile* (come il browser dell'utente o l'URL) *senza ricalcolare* o verificare quelle informazioni in un ambiente sicuro (il server).

☰ Esempio

Un URL come `www.example.com?qy=10&p=20&t=200` sul quale non viene effettuato alcun controllo lato server per assicurarsi che il totale sia effettivamente $t = p \times qy$. Un attaccante può facilmente modificare il parametro `t` nell'URL e pagare meno.

- **Contromisure:** La soluzione è la **mediazione completa**, ottenibile implementando e rispettando rigorosamente il concetto del **Reference Monitor** che ha il compito esclusivo di far rispettare le policy di controllo degli accessi.
 - **Sempre Invocato** (Completezza): Deve essere letteralmente impossibile da bypassare. Ogni singola richiesta di accesso a un oggetto deve passare attraverso di esso. Non ci sono "scorciatoie" per gli amministratori, né controlli saltati per motivi di performance.

- A *prova di manomissione* (Tamper-proof): Nessun utente, né un processo esterno, deve poter modificare le regole del monitor, disabilitarlo o alterarne il funzionamento. Deve essere protetto dal resto del sistema.
- *Verificabile* (Small and Testable): Deve essere un componente sufficientemente piccolo, semplice e isolato da poter essere analizzato matematicamente o testato in modo esaustivo per dimostrare che non contiene bug.

Dal momento del controllo al momento dell'utilizzo (TOCTTOU)

Il TOCTTOU nasce dal fatto che nei sistemi moderni le *operazioni non avvengono istantaneamente, ma richiedono microscopici intervalli di tempo*. Il difetto si consuma in due momenti distinti:

1. **Time of Check (ToC)**: Il sistema verifica se l'utente ha i permessi per compiere l'azione o se i dati sono validi.
2. **Time of Use (ToU)**: Il sistema esegue l'azione basandosi sul presupposto che il controllo effettuato al ToC sia ancora valido.

La vulnerabilità: Tra il momento 1 e il momento 2 esiste una **finestra temporale** (anche solo di pochi millisecondi). Se un attaccante riesce a modificare la risorsa *esattamente in quella finestra*, il sistema userà la risorsa modificata (malevola) convinto di aver già verificato la risorsa legittima.

Le Contromisure (Come si risolve?)

Le soluzioni mirano a "chiudere" completamente quella finestra temporale o a rendere inutile qualsiasi modifica effettuata dall'attaccante.

Operazioni Atomiche (Integrità Seriale)

L'atomicità (dal greco *atomo*, indivisibile) significa che il controllo (Check) e l'uso (Use) vengono fusi in **un'unica operazione indivisibile**. Il sistema operativo garantisce che nessun altro processo possa inserirsi nel mezzo.

- **Esempio pratico:** Invece di fare prima `controlla_se_esiste()` e poi `apri_file()`, si usano flag di sistema come `open(..., O_CREAT | O_EXCL)` che eseguono il controllo e l'apertura in un solo colpo sicuro a livello di Kernel.

Copia in Memoria Protetta (Isolamento)

Se il sistema deve verificare dei dati passati dall'utente (es. una struttura dati o un puntatore), la primissima cosa da fare è **copiare immediatamente quei dati nello spazio di memoria protetto del Kernel** (o del server) prima di fare qualsiasi controllo.

- Una volta copiati lì dentro, l'attaccante non ha più i permessi per modificarli. Il sistema può fare il Check e il ToU in totale tranquillità sulla copia sicura.

Sigillo Crittografico (Firme Digitali/HMAC)

Se i dati devono per forza transitare o risiedere in zone non protette tra il controllo e l'uso, il sistema applica una firma crittografica ai dati al momento del Check.

- Al momento del ToU, il sistema verifica se il sigillo è ancora integro. Se l'attaccante ha modificato anche solo un bit nella finestra temporale, il sigillo risulterà corrotto e l'azione verrà bloccata.

Punto di Accesso Non Documentato (Backdoor)

Durante lo sviluppo di un programma, il programmatore potrebbe creare un punto di accesso non documentato per favorire e velocizzare i test, dimenticando poi di rimuoverlo in produzione. Questo punto di accesso nascosto è detto **Backdoor** o **Trapdoor**.

- **Protezione:** Idealmente le backdoor dovrebbero essere individuate durante le revisioni del codice (Code Review). Tuttavia, non essendo documentate, potrebbero sfuggire facilmente ai revisori. Inoltre, spesso vengono aggiunte in un secondo momento, durante la fase di testing o di manutenzione.

Race Condition

Una **Race Condition** (condizione di corsa) si verifica quando due o più processi concorrenti vogliono accedere allo stesso tempo alla stessa risorsa condivisa.

Sono vulnerabilità molto difficili da individuare e correggere in quanto il loro manifestarsi è situazionale e dipende da svariati fattori e tempistiche di esecuzione.

Possono causare risultati incoerenti, indesiderati ed errati, compromettendo gravemente l'integrità e l'affidabilità del sistema.

Malware

Il termine **Malware** (Malicious Software) racchiude diverse categorie di programmi progettati per infiltrarsi o danneggiare un sistema.

- **Virus:** Programma che si replica infettando altri file legittimi (modificandone il codice). Può essere *transitorio* (si disattiva quando il programma ospite viene chiuso) o *residente* (rimane attivo nella memoria RAM).
- **Worm:** A differenza del virus, è autonomo e non ha bisogno di infettare altri file. Si autoreplica e si diffonde sfruttando le reti.
- **Trojan Horse:** Programma che si presenta con una funzione utile e innocua, ma nasconde un payload malevolo (es. apre una backdoor).
- **Altri tipi:** Spyware (spionaggio), Ransomware (sequestro dei dati tramite crittografia), Logic/Time Bomb (si attivano al verificarsi di un evento o a una data specifica), Rabbit (esaurisce le risorse di sistema replicandosi all'infinito).

Tipologie di Danno

1. **Non distruttivo:** Fastidi o scherzi (es. finestre pop-up, suoni anomali).

2. **Distruttivo:** Corruzione di file, blocco del sistema operativo, formattazione del disco.
3. **Commerciale/Criminale:** Furto di dati sensibili (credenziali, carte di credito), spionaggio industriale o inserimento in una botnet.

La difficoltà di rimozione: I malware moderni sono spesso **multipartiti**. L'infezione è distribuita in vari punti del sistema; se l'antivirus elimina solo una parte, i frammenti rimanenti possono reinstallare l'intero malware.

Vettori di Infezione e Propagazione

Per fare danni, il malware **deve essere eseguito**. Gli attaccanti usano vari vettori per ingannare il sistema o l'utente.

Come entra nel sistema

- **Installer contraffatti:** Il malware è nascosto nel pacchetto di installazione di un software apparentemente legittimo scaricato da internet.
- **Allegati ed Ingegneria Sociale:** File inviati via mail che richiedono l'interazione dell'utente per essere aperti.
- **Macro e Documenti:** Il codice malevolo è nascosto all'interno di documenti (es. Word, Excel, PDF) che supportano l'esecuzione di script (macro).
- **Autorun / Esecuzione Automatica:** Il malware si inserisce nelle chiavi di registro o nei file di configurazione per essere eseguito automaticamente a ogni avvio del Sistema Operativo.

Come si lega al codice legittimo (Infezione)

Gli autori frammentano il malware e lo legano ai programmi sani in tre modi:

1. **Aggiunta (Append):** Il virus inserisce il proprio codice all'inizio o alla fine del file eseguibile legittimo.
2. **Circondamento:** Il virus inietta se stesso sia prima che dopo il programma originale, mantenendo il controllo assoluto (pre e post esecuzione).
3. **Sostituzione (Integrazione):** Sovrascrive fisicamente parti del codice originale. Richiede una conoscenza profonda dell'eseguibile bersaglio.

Come prende il controllo

- **Modifica dei puntatori:** Altera la File Table del sistema affinché, quando l'utente clicca su un programma, il sistema operativo esegua invece il virus.
- **Boot Sector:** Infetta il settore di avvio del disco. Il malware viene caricato **prima** del Sistema Operativo e dell'antivirus, ottenendo privilegi massimi e risultando invisibile.
- **Residenza in memoria:** Si aggancia a servizi di sistema sempre attivi, garantendosi un'esecuzione continua e la possibilità di infettare ogni nuovo dispositivo collegato.

Furtività (Stealth) e Tecniche di Evasione

L'obiettivo principale di un malware avanzato è non farsi scoprire dagli scanner antivirus, che solitamente cercano pattern noti (le *firme* o *signature*).

Tecniche di offuscamento statico:

- Riorganizzare l'ordine dei moduli o delle istruzioni.
- Inserire **Dead Code** (codice inutile come `a = a;` o stringhe casuali) per cambiare l'impronta del file.
- Sostituire istruzioni assembly con equivalenti logici.
- **Steganografia**: Nascondere il codice malevolo nei bit meno significativi di file multimediali grandi (es. immagini o video).
- **Spoofing del File Type**: Sfruttare intestazioni anomale nei file per ingannare il sistema su quale applicazione debba aprire il documento.

Virus Polimorfi e Crittografati

- **Polimorfismo**: Il virus muta il proprio aspetto (codice) a ogni nuova infezione, riposizionando variabili e istruzioni, pur mantenendo intatta la sua logica distruttiva.
- **Virus Crittografati**: Il corpo del virus è illeggibile. Il file è composto da 3 parti: una *chiave*, il *codice cifrato*, e una piccola *routine di decifratura*. Poiché la routine di decifratura deve essere in chiaro per funzionare, gli antivirus usano quella come "firma" da cercare.

Contromisure

Contromisure per gli Utenti (Vigilanza dell'utente)

- Utilizzare solo software commerciale, acquistato da fornitori affidabili;
- Testare tutto il nuovo software su un computer isolato;
- Aprire allegati e altri file potenzialmente infetti solo quando si è certi della loro sicurezza;
- Installare software e altri file eseguibili potenzialmente infetti solo quando si è certi della loro sicurezza;
- Tenere presente che ogni sito web può essere potenzialmente dannoso;
- Creare un'immagine di sistema ripristinabile e conservarla in un posto sicuro;
- Creare e conservare copie di backup dei file di sistema eseguibili;
- Applicare tutte le patch tempestivamente, tranne quando ciò causerebbe più danni che benefici;
- Utilizzare scanner antimalware.

Rilevatori di virus

Gli **scanner antivirus** sono strumenti che cercano segni di infezione e codice dannoso, cercando una firma o un'impronta digitale. Questi strumenti hanno però due limitazioni:

1. Sono retrospettivi, ricercano infezioni note richiedendo aggiornamenti costanti e rimanendo vulnerabili a nuovo codice malevolo;
2. I pattern sono statici e cercano di essere lunghi per evitare di classificare erroneamente programmi legittimi. Di conseguenza, gli autori di virus variano il loro codice.

Analisi del codice: È complessa in quanto si dispone solo del codice macchina finale e spesso solo in minima parte. Benché lo si possa convertire con un disassembler in assembly, determinarne l'impatto resta complesso.

Modelli di memorizzazione: Un virus può attaccarsi ad un file, aumentandone la dimensione, oppure può cancellare parte del programma e sostituirlo, compromettendone però il funzionamento. Uno scanner può usare un codice o un checksum per rilevare le modifiche o cercare pattern sospetti.

Ingegneria del Software per la Sicurezza

Per prevenire le vulnerabilità, gli sviluppatori devono seguire principi rigorosi di Ingegneria del Software.

Modularità

Suddividere il software in piccoli componenti (moduli). Un buon modulo ha **Alta Coesione** (fa una sola cosa in modo logico) e **Basso Accoppiamento** (dipende pochissimo dagli altri moduli).

- **Vantaggi:** Facilita manutenzione, test, comprensione e isolamento degli errori.
- **Regola d'oro:** Ogni modulo deve essere monouso, piccolo, semplice e indipendente.

Principi di Progettazione

Principio	Descrizione
Incapsulamento	Nasconde i dettagli implementativi complessi dietro interfacce ben definite.
Information Hiding	Il modulo è una "scatola nera". Gli altri moduli non sanno come lavora internamente, sanno solo come passargli input e ricevere output.
Sospetto Reciproco	Nessun processo si fida ciecamente di un altro. Ogni modulo valida rigorosamente i dati in ingresso (Sanitization).
Confinamento (Sandboxing)	Il Sistema Operativo isola un processo sospetto, limitando le risorse (file, rete) a cui può accedere.
Diversità Genetica	Evitare l'omologazione (es. usare software o librerie diverse per scopi simili) aumenta la resilienza globale, anche se costa di più.

Il Testing e i suoi Limiti

Il testing è diviso in fasi sequenziali. Può essere **Black-box** (si testa solo l'input/output senza vedere il codice) o **Clear-box/White-box** (si testa conoscendo la struttura interna).

Fase di Test	Cosa verifica
1. Unit Testing	Il singolo modulo o funzione in isolamento.
2. Integration	Che i vari moduli comunichino correttamente tra loro.
3. Function	Che il sistema rispetti le specifiche tecniche dei requisiti.
4. Performance	I tempi di risposta, lo stress e l'uso dell'hardware.
5. Acceptance	Se il software soddisfa i bisogni reali del cliente finale.
6. Installation	La corretta configurazione nell'ambiente di produzione reale.
7. Regression	Che l'aggiunta di una patch o funzionalità non abbia "rotto" cose che prima funzionavano.

I limiti del Testing (Principio di Dijkstra): I test possono dimostrare *la presenza* di difetti, ma non potranno mai dimostrare *l'assenza assoluta* di guasti. Esaurire tutte le combinazioni possibili di input e stati di memoria è matematicamente impossibile.

Sicurezza Architeturale e Pratiche

Principi di Sicurezza (Saltzer & Schroeder)

1. **Minimo Privilegio:** Dare a un utente/processo solo i permessi strettamente necessari per il tempo necessario.
2. **Economia del Meccanismo:** Più un sistema di sicurezza è semplice, meno probabilità ha di contenere bug.
3. **Progettazione Aperta (Open Design):** La sicurezza non deve dipendere dalla segretezza del codice (no *Security by Obscurity*), ma dalla robustezza delle chiavi/password.
4. **Mediazione Completa:** Ogni singolo accesso deve essere validato dal sistema, senza eccezioni o cache non sicure.
5. **Fail-safe Defaults:** L'accesso di default deve essere negato. I permessi vanno concessi esplicitamente.
6. **Separazione dei Privilegi:** Richiedere l'approvazione di due o più condizioni per operazioni critiche.
7. **Facilità d'uso:** Se la sicurezza è troppo complessa, gli utenti troveranno il modo di aggirarla.

Verifica vs Validazione

- **Verifica:** Dimostrazione formale (matematica) che il codice fa esattamente ciò per cui è stato progettato senza errori logici. (*Stiamo costruendo il prodotto nel modo giusto?*)
- **Validazione:** Accertamento empirico (tramite test e review) che il software soddisfi i requisiti operativi richiesti dal committente. (*Stiamo costruendo il prodotto giusto?*)

Approcci Fallimentari (Da evitare)

- **Penetrate & Patch:** Rilasciare software vulnerabile e mettere "toppe" (patch) solo quando si scoprono le falle. Risolve il sintomo, ma l'architettura resta debole e le patch spesso creano nuovi bug (regressione).
- **Sicurezza tramite Oscurità:** Credere che nascondere il codice sorgente o il funzionamento del sistema tenga lontani gli hacker. Il reverse engineering vanifica sempre questo approccio.

Sicurezza Internet

Attacchi al Browser

Il browser è il bersaglio principale per rubare informazioni, installare malware o negare l'accesso ai servizi. Gli attaccanti possono colpire in quattro modi:

1. Compromettendo il Sistema Operativo (SO) per sabotare il browser.
2. Intercettando o alterando il traffico di rete da e verso il browser.
3. Attaccando i componenti interni del browser stesso.
4. Modificando i contenuti legittimi di un sito web.

Tipi di Attacchi al Browser

- **Man-in-the-Browser:** Un malware infetta il browser e monitora, copia o altera tutto ciò che l'utente digita o visualizza, rubando credenziali e dati bancari.
- **Keystroke Logger (Keylogger):** Software o hardware che intercetta e registra ogni singolo tasto premuto sulla tastiera, inviando i log all'attaccante.
- **Page-in-the-Middle:** L'utente viene reindirizzato in modo invisibile su una pagina controllata dall'attaccante (es. clicca su "Login" in un sito vero e finisce su un modulo di login falso).
- **Sostituzione del Download:** Spesso legata al Page-in-the-Middle. L'utente scarica un software apparentemente legittimo a cui è stato iniettato del codice malevolo.
- **User-in-the-Middle:** L'utente viene usato a sua insaputa per risolvere test di sicurezza (come i CAPTCHA) per conto di bot automatizzati.
 - **Evoluzione dei CAPTCHA:** Nati come lettere distorte (superati dagli OCR), poi passati a calcoli matematici (superati dai bot), fino al reCAPTCHA (riconoscimento immagini) e ai moderni sistemi invisibili che analizzano le micro-interazioni dell'utente (movimento mouse, tempi di reazione).
 - **Private Access Token (PAT):** Introdotto da Apple nel 2022. Sostituisce il CAPTCHA visivo: il dispositivo attesta l'autenticità dell'utente analizzandone il comportamento in background e inviando un token monouso crittografico al sito web.

Errori di Identificazione e Autenticazione

Il tallone d'Achille degli attacchi "in-the-middle" è la fase di identificazione. Se un malware ruba i file di autenticazione (certificati, cookie, chiavi) usati per il login automatico, può bypassare le difese.

Punti Vulnerabili

- **Attacco Replay:** L'attaccante intercetta i pacchetti di rete contenenti le credenziali crittografate, li salva e li re-invia (riproduce) in seguito per ottenere l'accesso.
- **Dirottamento della Sessione (Session Hijacking):** L'attaccante aspetta che l'utente effettui un login legittimo e poi ne "ruba" il cookie di sessione per operare a suo nome.

Tecniche di Autenticazione Efficaci

Per bilanciare sicurezza e usabilità, i sistemi moderni usano:

- **Segreto Condiviso:** Un'informazione nota solo a client e server (es. timestamp dell'ultimo accesso).
- **One Time Password (OTP):** Password temporanee usa e getta.
- **Out-of-band Communication (2FA):** La verifica avviene su un canale di comunicazione fisicamente o logicamente separato (es. notifica push sullo smartphone mentre navighi da PC).
- **Continuous Authentication:** La sessione viene verificata e cifrata continuamente in background.

Attacchi Rivolti agli Utenti

Protezione dei siti da modifiche dannose

Esistono diversi modi, oltre alla crittografia che qui non è adatta, per proteggere i siti web.

- **Integrity Checksum:** Serve a capire se un sito è stato modificato o alterato dopo che l'attacco è avvenuto. Un algoritmo matematico analizza un file e genera un valore hash. Se il file viene alterato, il suo hash cambia inevitabilmente.
- **Codice/Dati Firmati:** Per proteggere l'utente, lo sviluppatore firma il codice del sito utilizzando un certificato digitale rilasciato da un'autorità fidata, chiamata CA (Certificate Authority). Quando il browser scarica quel codice, ne verifica matematicamente la firma.

Contenuti Web Dannosi

Attacchi tramite alterazione visiva e inganno

- **Website Defacement:** L'attaccante viola il server e sostituisce la home page per vandalismo o attivismo (alterazione grafica).
- **Fake Website (Phishing):** Creazione di un sito clone per indurre l'utente a inserire i propri dati sensibili.

- **Fake Code:** Distribuzione di script o programmi contenenti malware nascosto.

Difese del Server: Per verificare che un sito non sia stato manomesso si usano l'**Integrity Checksum** (l'alterazione di un file ne cambia matematicamente l'hash) e il **Codice Firmato** (il browser scarica il sito e ne verifica la firma digitale emessa da una Certificate Authority fidata).

Contenuti Dannosi Integrati

- **Web Bug (Tracking Pixel):** Immagini invisibili (1x1 pixel) nascoste nelle pagine web o nelle mail. Quando il browser le carica, invia dati al server remoto (es. IP, orario), permettendo la profilazione e il tracciamento incrociato.
- **Clickjacking:** Sovrapposizione di un layer invisibile sopra un pulsante legittimo. L'utente crede di cliccare sul sito, ma in realtà innesca un'azione dannosa nel layer nascosto (es. approva un pagamento).
- **Drive-by Download:** Il solo atto di visitare una pagina compromessa innesca il download e l'esecuzione silente di un malware, sfruttando le vulnerabilità del browser.

Acquisizione di Dati dell'Utente o del Sito Web

Questi attacchi non colpiscono l'utente finale direttamente, ma la logica del server e la mancata sanificazione degli input.

Cross-Site Scripting (XSS)

Iniezione di codice eseguibile (spesso JavaScript) all'interno di una pagina web fornita ad altri utenti.

- **Riflesso (Reflected):** Il codice è inserito nell'URL e viene eseguito solo per l'utente che clicca quel link specifico.
- **Persistente (Stored):** Più grave. Lo script viene salvato nel database del server (es. in un commento di un forum) ed eseguito sul browser di *tutti* gli utenti che visiteranno quella pagina.

SQL Injection (SQLi)

Manipolazione dei campi di input per inviare query dannose al database SQL del server.

- **Inband:** Estrae i dati usando lo stesso canale della richiesta (es. bypass del login con tautologie tipo `OR 1=1`, o uso di query accodate/piggybacked).
- **Inferential (Blind):** Il server non restituisce dati espliciti, ma l'attaccante deduce informazioni osservando il comportamento dell'applicazione (es. differenze nei tempi di risposta o messaggi di errore) inviando query vere o false.
- **Out-of-band:** I dati rubati vengono inviati all'attaccante attraverso un canale diverso (es. costringendo il database a fare una richiesta DNS verso un server malevolo).

Vulnerabilità di Path e Server

- **Directory Traversal (Dot-Dot-Slash):** L'attaccante manipola l'URL (inserendo `../`) per uscire dalla cartella pubblica del web server e accedere a file critici del SO (es. `/etc/passwd`).
- **Server-Side Include (SSI):** Sfrutta la possibilità di inserire comandi speciali (es. `exec`) nei form web. Se il server li interpreta incautamente, esegue codice arbitrario sulla macchina ospite.

App Mobile e Sicurezza

Differenze tra App e Siti Internet

Caratteristica	App Mobile	Siti Internet
Residenza	Risiede fisicamente sul dispositivo	L'accesso richiede una connessione al server
Interazione	Interagisce profondamente con HW e SW	Eseguito in sandbox tramite un browser
Esecuzione	Può runnare in background	Stop esecuzione quando ci si disconnette
Dati	Colleziona, immagazzina e trasmette dati	È possibile ripulire facilmente tutti i dati (cache/cookie)
Aggiornamenti	Solo se connesso a internet e se l'utente accetta	Può aggiornare il codice all'istante lato server

Cause delle vulnerabilità nelle App

Le app sono bersagli redditizi per l'enorme bacino di utenza. Spesso presentano falle perché:

1. **Permission Creep:** Accumulano permessi eccessivi aggiornamento dopo aggiornamento.
2. **Scatola Nera:** Sono software compilati (opachi) che nascondono il loro reale funzionamento.
3. **Priorità al Business:** Se l'app è gratis, il business model è la vendita dei dati (lavorano per chi le crea, non per l'utente).
4. **UX vs Sicurezza:** I budget vengono spesi per l'interfaccia grafica, trascurando la sicurezza

Movente, Opportunità e Metodo

- **Movente:** Guadagno economico (soldi).
- **Opportunità:** L'enorme bacino di utenti con tante app installate su tanti dispositivi implica che anche un attacco con un bassissimo tasso di successo può risultare molto redditizio.
- **Metodo:** Le app possono nascondere vulnerabilità in modo molto efficace, facendo sì che né le *code review* né l'esecuzione rendano l'attacco immediatamente evidente.

Vulnerabilità derivanti dall'utilizzo delle App (OWASP Top 10 Mobile)

L'Open Web App Security Project (OWASP) pubblica periodicamente i rischi per la sicurezza più critici.

- **Injection:** La sicurezza contro le injection dipende interamente dal programmatore, che deve sanitizzare ogni singolo dato inserito dall'utente. Per un revisore è quasi impossibile garantire che questa verifica sia stata eseguita ovunque nel codice.
- **Cross-Site Scripting (XSS) su Mobile:** Lo script malevolo passa da un sito al dispositivo dell'utente tramite l'app vulnerabile, consentendo l'accesso a qualsiasi risorsa per cui l'app abbia i permessi.
- **Autenticazione Errata:** Gli sviluppatori potrebbero implementare sistemi di login deboli, aggirabili o vulnerabili al bruteforce.
- **Errore nel controllo degli accessi:** I rischi includono la mancata applicazione del principio del minimo privilegio, consentire l'elusione dei controlli, l'elevazione dei privilegi o la mancata revoca dei token al termine di una sessione.
- **Riferimenti diretti a oggetti non sicuri (IDOR):** Un'app espone riferimenti diretti e prevedibili a risorse interne senza verificarne l'autorizzazione (es. iterare su un URL come `.../doc.php?id=00001` cambiando i numeri finali).
- **Registrazione e monitoraggio insufficienti:** Un'app potrebbe non tracciare correttamente l'attività, rendendo impossibile capire in fase di incident response quali dati siano stati compromessi.
- **Vulnerabilità crittografiche:** Un'app potrebbe non proteggere (o proteggere con algoritmi deboli) i dati sensibili in transito sulla rete o in memoria locale, esponendoli ad intercettazioni.
- **Problemi di integrità del software e dei dati:** L'infrastruttura potrebbe permettere la manomissione dei dati o persino la sostituzione fraudolenta del codice dell'applicazione stessa.
- **Componenti vulnerabili o obsoleti:** Uso di plugin, librerie di terze parti o moduli non più supportati o con CVE (Common Vulnerabilities and Exposures) note.
- **Configurazione di sicurezza errata:** Mancato aggiornamento o hardening delle configurazioni di app, server e dispositivi.

Perché le app hanno difetti

- **Hanno potere quasi illimitato:** Il SO chiede all'utente di concedere permessi molto ampi, che tendono ad accumularsi dopo ogni aggiornamento (*permission creep*).
- **Sono opache e incontrollabili:** Un'app è una scatola nera (compilata) in cui ci potrebbero essere innumerevoli funzionalità nascoste.
- **Lavorano per chi le ha create, non per l'utente:** Il famoso adagio "se un'app è gratuita il prodotto sei tu" (monetizzazione dei dati).
- **Gli utenti preferiscono l'impatto visivo alla sicurezza:** Di conseguenza, chi sviluppa app investe la maggior parte del budget sulla grafica e sull'usabilità (UX/UI) e pochissimo sulla sicurezza.

Sviluppo di App Sicure

Qualifiche degli sviluppatori

Non esiste alcun ente ufficiale che certifichi in modo universale gli sviluppatori qualificati, e lo sviluppo di app non richiede titoli accademici particolari. L'unico reale indicatore di affidabilità resta la presenza di un rigoroso processo aziendale regolamentato che includa la sicurezza *by design* (fin dalla progettazione).

Controlli automatizzati dello sviluppo

Metodologie moderne come DevOps e Agile puntano in modo aggressivo sulla velocità di rilascio, riducendo la documentazione e sollevando il timore che tale rapidità penalizzi la sicurezza.

La mentalità orientata alla sicurezza è fondamentale: bisogna anticipare come un attaccante potrebbe far fallire il software. È diventato standard introdurre l'automazione tramite strumenti di verifica continua della configurazione e sistemi di autoprotezione a runtime, tramite specifici sensori che i programmatori inseriscono nel codice.

Strumenti di Analisi del Codice

Esistono 2 grandi categorie per l'analisi:

- **SAST (Static Application Security Testing):** Ispezionano il codice sorgente (o il bytecode) a riposo alla ricerca di errori di programmazione prima della compilazione.
- **DAST (Dynamic Application Security Testing):** Testano l'applicazione effettivamente in esecuzione, simulando attacchi esterni.
- **IAST (Interactive Application Security Testing):** Questi tool combinano SAST e DAST in un controllo simultaneo reso possibile tramite *hook* (sonde) inseriti direttamente nel software.
- **RASP (Runtime Application Self-Protection):** È un agente integrato per l'autoprotezione che monitora costantemente gli input e il comportamento dell'app per bloccare in tempo reale eventuali minacce.

Attacchi via Mail e Messaggi

Spam e Phishing

Mail ingannevoli che simulano enti legittimi. Nonostante i filtri blocchino la maggior parte dello spam, i costi di invio rasentano lo zero, rendendo profittevole anche una percentuale infinitesimale di "vittime".

- **Scopi:** Diffusione malware, furto credenziali, *Pump and dump* (manipolazione azionaria tramite fake news), pubblicità aggressiva.
- **Mitigazione:** Difficile per legge per via dello spoofing (falsificazione del mittente). Le soluzioni tecniche includono il blocco del rate-limiting a livello di rete o l'introduzione della *Proof of Work* (costo computazionale per ogni invio).

Protezione Crittografica delle Mail

S/MIME (Secure/Multipurpose Internet Mail Extensions):

Lo standard aziendale. Usa un'infrastruttura a chiave pubblica (PKI) centralizzata. Le chiavi e i certificati sono garantiti da un'Autorità di Certificazione (CA) gerarchica e fidata.

PGP (Pretty Good Privacy):

Usa un modello decentralizzato (Rete di Fiducia / *Web of Trust*), dove gli utenti firmano reciprocamente le proprie chiavi pubbliche. Il processo per l'invio di una mail sicura è il seguente:

- **Firma Digitale:**
Garantisce autenticità e integrità
Genera l'hash del messaggio originale e lo cifra usando la chiave privata del mittente.
- **Generazione della chiave di sessione:**
Crea una chiave simmetrica monouso (casuale) per questa specifica comunicazione.
- **Cifratura del Messaggio:**
Cifra l'intero corpo dell'email (e la firma) utilizzando la chiave di sessione appena generata.
- **Cifratura della chiave e invio:**
Garantisce la confidenzialità
Cifra la chiave di sessione usando la chiave pubblica del destinatario. Pacchetto finale e messaggio cifrato vengono inviati.

Sicurezza del Sistema Operativo

Il Sistema Operativo (SO) rappresenta la prima linea di difesa informatica, incaricato di isolare gli utenti, proteggere la memoria, gestire l'autenticazione e le risorse hardware. Proprio per questo motivo, esso è il principale bersaglio di attacchi, poiché comprometterlo significherebbe avere il completo controllo della macchina.

Funzioni del SO che coinvolgono la sicurezza informatica

Il SO implementa diverse funzioni critiche per mantenere il sistema sicuro e stabile:

- **Condivisione forzata:** Garantisce che le risorse siano disponibili a più utenti mantenendone l'integrità.
- **Comunicazione e sincronizzazione interprocesso:** Regola in modo sicuro come processi diversi "parlano" tra loro e si coordinano per accedere alle risorse condivise.
- **Protezione dei dati critici:** Difende i file sensibili e le strutture dati utilizzate dal SO stesso.
- **Garanzia di servizio equo (Fairness):** Evita il *Denial of Service* (DoS) accidentale o doloso, impedendo che un singolo processo monopolizzi le risorse (es. CPU o RAM).
- **Interfaccia verso l'hardware:** Agisce da intermediario obbligatorio tra i programmi utente e i componenti fisici.
- **Autenticazione utente:** Gestisce le procedure di login e verifica delle identità.
- **Protezione della memoria:** Isola lo spazio RAM di ciascun processo per evitare letture/scritture non autorizzate.

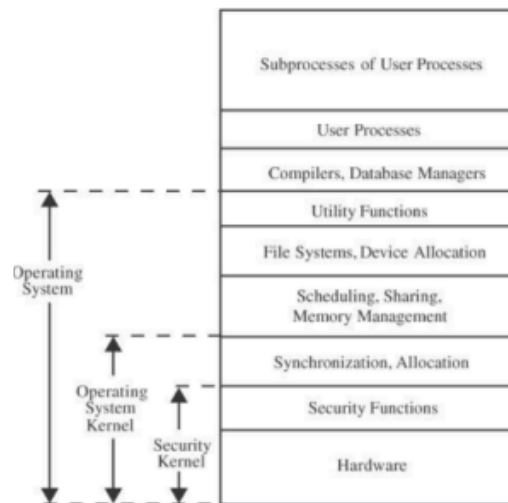
- **Controllo degli accessi (I/O e File):** Vigila su chi può leggere, scrivere o eseguire determinati file e periferiche.

Oggetti Protetti

Il SO deve proteggere: la **Memoria**, i **Dispositivi I/O** (condivisibili come i dischi, o riutilizzabili in serie come le stampanti), i **Programmi/Sottoprocedure**, le **Reti** e i **Dati condivisibili**.

L'Architettura di Protezione Per difendere questi oggetti, l'architettura del SO viene spesso rappresentata visivamente in due modi:

- **A strati verticali (Layered view).**
- **A cerchi concentrici (Ring architecture):** I livelli più interni (i **Ring** centrali) ospitano le funzioni più critiche. Questo nucleo interno inviolabile prende il nome di **Security Kernel**.



Strumenti del SO per Implementare Funzioni di Sicurezza

Poiché un SO deve spesso girare su hardware generico (che non ha protezioni fisiche integrate), utilizza meccanismi puramente software per difendersi anche dai propri moduli o add-on:

1. **Virtualizzazione:** Il SO crea una *Virtual Machine (VM)* isolata per l'utente, astruendo l'hardware fisico. L'utente vede solo le risorse a cui è autorizzato. Questo sistema è governato dall'**Hypervisor**, un software che intercetta e filtra ogni singola istruzione in uscita dalla VM.
2. **Sandbox:** Un ambiente di test isolato e "recintato". I programmi non fidati (es. codice scaricato da internet) vengono eseguiti qui dentro: anche se sono malevoli, non possono "evadere" per danneggiare il resto del SO. (Concetto nato originariamente con Java).
3. **Honeypot (Vaso di miele):** La regola d'oro dell'honeypot è che *non ha alcuna funzione di produzione legittima*. Non ospita servizi reali e gli utenti normali non hanno alcun motivo per interagirci. Di conseguenza, *qualsiasi tentativo di connessione, scansione o accesso a un honeypot è considerato per definizione malevolo o altamente sospetto*.

Un *honeypot* viene schierato per tre motivi principali:

- **Distrazione (Diversion):** Attira l'attenzione dell'attaccante, facendogli sprecare tempo e risorse preziose su finte vulnerabilità, allontanandolo dai server reali (i *target* critici).
- **Rilevamento precoce (Tripwire):** Funge da allarme silenzioso. Spesso viene posizionato all'interno della rete aziendale. Se un hacker supera il firewall e inizia a scansionare la rete

interna, incapperà nell'honeypot, allertando immediatamente gli amministratori.

- **Raccolta di Intelligence:** È l'obiettivo più importante. L'honeypot registra ogni singola mossa dell'attaccante: quali porte scansiona, quali password prova, quali malware installa e quali tecniche usa (TTP - *Tactics, Techniques, and Procedures*). Questo permette agli analisti di studiare il nemico e aggiornare le difese dei server reali di conseguenza.

Separazione e Condivisione

La sicurezza si basa sul concetto di **Separazione** (che può essere *Fisica*, *Temporale*, *Logica* o *Crittografica*). Tuttavia, separare tutto azzerà l'efficienza. Il SO deve quindi bilanciare la separazione con la **Condivisione**, applicando diversi livelli di granularità:

- **Non proteggere:** Usato quando processi sensibili avvengono in momenti fisicamente o temporalmente separati.
- **Isolare:** I processi girano in contemporanea, ma il SO li nasconde l'uno all'altro (non sanno di non essere soli nel sistema).
- **Tutto o nulla:** Un oggetto è dichiarato interamente pubblico o rigorosamente privato.
- **Condividere ma limitare l'accesso:** Si utilizzano le Access Control List (ACL) per decidere *chi* può accedere.
- **Limitare l'uso:** Si restringe non solo *chi* accede, ma *come* (es. puoi leggere il file, ma non puoi stamparlo).

Protezione Hardware della Memoria

Per isolare i processi in modo rapido, il SO si affida a registri e meccanismi forniti direttamente dal processore (hardware):

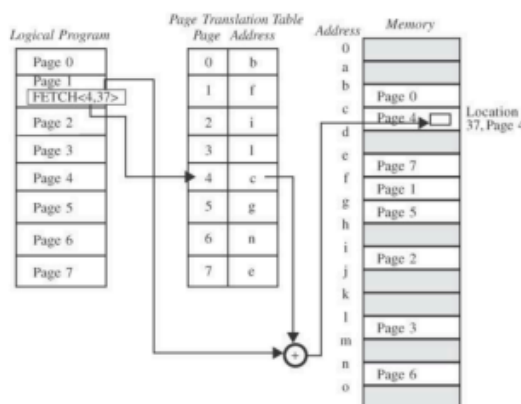
- **Fence (Recinto):** Il metodo più rudimentale. Crea un confine fisso in memoria. Si è evoluto nel *Fence Register* (Address Limit Register), che permette di spostare dinamicamente il confine tra lo spazio protetto del SO e quello dell'utente. Resta però un limite singolo: divide solo il SO dai programmi, non i programmi tra loro.
- **Registri Base/Bound:** Usano una coppia di registri. Il *Base* indica dove inizia il processo, il *Bound* dove finisce. Ad ogni *Context Switch*, il SO aggiorna questi registri. I sistemi avanzati usano due coppie di registri per separare ulteriormente il codice del programma dai suoi dati.
- **Tagged Architecture:** Aggiunge bit extra (tag) ad ogni singola cella di memoria a livello hardware, per definire in modo matematico e inalterabile i diritti di accesso di quel blocco.

Gestione della Memoria Virtuale

La segmentazione è una tecnica di memoria virtuale che divide un programma in blocchi logici di dimensioni variabili (i segmenti). Ogni indirizzo viene espresso come una coppia `<nome segmento, offset>` e tradotto in memoria fisica tramite una tabella dei segmenti. In questo modo, il SO controlla ogni accesso a determinati segmenti e può assicurarsi che l'offset richiesto non superi la lunghezza consentita del segmento.

- **Paging**

Il paging divide i programmi in pagine (blocchi di dimensione fissa) e la memoria fisica in unità uguali dette *page frames*. L'indirizzo è espresso come `<pagina, offset>` e viene tradotto tramite una **tabella delle pagine**.



- **Segmentazione Paginata**

Questa tecnica unisce i vantaggi dei due metodi precedenti: il programma viene prima diviso in segmenti logici, i quali vengono poi frammentati in pagine di dimensione fissa per ottimizzare l'efficienza della memoria.

Sicurezza nella Progettazione dei Sistemi Operativi

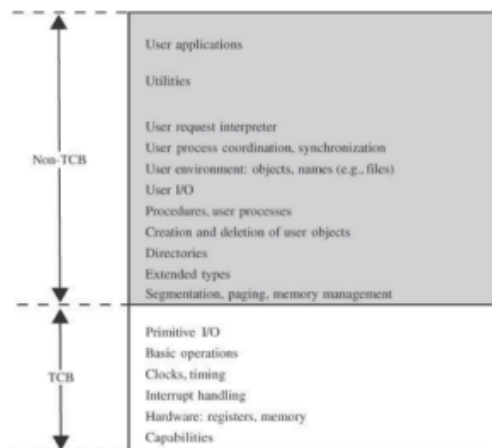
Un SO sicuro non nasce per caso, ma segue regole architetturali precise:

1. **Semplicità:** Meno codice c'è, meno bug ci sono e più è facile fare i test.
2. **Progettazione a Strati:** I livelli interni (Kernel) hanno permessi massimi, quelli esterni (User) permessi minimi.
3. **Microkernel:** Spostare tutte le funzioni critiche di sicurezza in un nucleo minuscolo, isolato e facile da validare matematicamente.
4. **Monitor di Riferimento:** Un punto di controllo centralizzato che intercetta e valuta ogni singola richiesta di accesso.
5. **Correttezza e Completezza:** Non ci devono essere eccezioni, porte sul retro o regole ignorate.
6. **Principi di base:** Applicare sempre la *Mediazione Completa* e il *Least Privilege* (Privilegio Minimo).

Trusted Computing Base (TCB)

I **Sistemi Fidati** sono SO certificati che garantiscono correttezza e integrità assoluta. Il cuore di questi sistemi è la **Trusted Computing Base (TCB)**: la vera e propria "fortezza inattaccabile". È l'insieme di tutto l'hardware, il software e il firmware strettamente necessari per imporre la politica di sicurezza.

- **Cosa fa la TCB:** Monitora le operazioni di I/O, la memoria, l'attivazione dei processi e i context switch.
- **Come si implementa:** Inserire tutto il SO nella TCB lo renderebbe un monolite impossibile da gestire. La soluzione è creare un piccolo **Security Kernel** (che fa parte della TCB) posizionato direttamente sopra l'hardware, e costruire il resto delle funzioni del SO al di fuori di esso.



Sicurezza di Rete

Tecniche di Intercettazione

L'intercettazione è l'accesso non autorizzato ai dati in transito. Le modalità dipendono strettamente dal mezzo fisico utilizzato per la comunicazione:

- **Sniffing:** Intercettazione passiva del traffico dati (pacchetti) che transita su una rete locale (LAN).
- **Radiazione:** Cattura delle emanazioni elettromagnetiche (leakage) generate naturalmente dai cavi in rame attraversati da corrente.
- **Cable Splicing:** Intercettazione fisica e invasiva. Il cavo (es. rame) viene letteralmente tagliato o "spellato" per collegarci un dispositivo hardware di ascolto (tap).

Mezzi Trasmissivi e Vulnerabilità

Ogni tecnologia di trasmissione presenta vantaggi logistici ma specifiche vulnerabilità alla sicurezza:

Mezzo Trasmissivo	Caratteristiche Principali	Livello di Vulnerabilità
Cavo in Rame	Economico, facile installazione e manutenzione.	Alta: Vulnerabile a <i>cable splicing</i> e intercettazione via radiazioni elettromagnetiche.
Fibra Ottica	Trasmette luce, non impulsi elettrici.	Minima: Intercettarla senza spezzare il cavo (e far cadere il segnale) è estremamente complesso.
Microonde	Segnale forte e direzionale. Richiede linea di vista perfetta (Line of Sight).	Media: Intercettabile posizionandosi fisicamente lungo il fascio direzionale.
WiFi / Radio	Diffusione capillare, comodità assoluta. Il segnale degrada con la distanza.	Molto Alta: Chiunque nel raggio di copertura dell'antenna può "ascoltare" i pacchetti.
Satellitare	Copertura eccellente per aree isolate, ma introduce forte latenza	Alta: Il segnale "piova" su aree geografiche vastissime, facilitando l'intercettazione.

Mezzo Trasmissivo	Caratteristiche Principali	Livello di Vulnerabilità
	(delay).	

Minacce alle Comunicazioni di Rete

Le minacce alle reti informatiche colpiscono i tre pilastri della sicurezza (Riservatezza, Integrità, Disponibilità):

1. **Intercettazione (Minaccia alla Riservatezza):** Accesso e lettura non autorizzata dei dati in transito.
 - *Contromisure:* **Crittografia** (la difesa primaria e più efficace), sicurezza fisica delle linee, routing controllato e linee dedicate.
2. **Modifica (Minaccia all'Integrità):** Alterazione o corruzione dei dati durante il viaggio.
3. **Interruzione (Minaccia alla Disponibilità):** Blocco totale o degradazione parziale delle risorse di rete.

Fattori che facilitano gli attacchi di rete

- **Anonimato:** Possibilità di mascherare l'origine dell'attacco (es. tramite *IP Spoofing*).
- **Molteplicità dei punti di attacco:** In una rete interconnessa, un singolo dispositivo vulnerabile può compromettere tutti gli altri.
- **Condivisione delle risorse:** Le difese di un singolo PC spesso non bastano a proteggere le cartelle o le risorse di rete condivise.
- **Complessità e Perimetro:** Le reti si espandono (cloud, smart working), rendendo indefinito il perimetro aziendale. I Sistemi Operativi e i router intermedi aggiungono complessità e vettori d'attacco.
- **Routing non controllabile:** L'utente invia un pacchetto ma non ha alcun potere decisionale sui nodi (router fisici) che questo attraverserà su Internet.

Modifica dei Dati (Data Corruption)

L'attaccante interviene attivamente nel flusso di comunicazione alterandone il contenuto. Si divide in 4 casistiche:

- **Sostituzione:** Scambio di una parte dei dati originali con dati malevoli. È molto facile nei protocolli non cifrati dove la struttura (i campi) del messaggio è nota.
 - *Contromisure:* Crittografia e controlli di integrità (es. funzioni di Hash).
- **Inserimento:** Iniezione di pacchetti o comandi falsi all'interno di un flusso legittimo in corso. Può avvenire anche alla cieca, senza decifrare il messaggio originale, se si conosce dove colpire.
- **Replay (Ripetizione):** L'attaccante "sniffa" dati validi (come le credenziali di accesso o un token) e li conserva per riutilizzarli in seguito, simulando di essere l'utente legittimo.
 - *Contromisure:* Uso di *Timestamp* (marche temporali di validità) e numeri di sequenza univoci.

- **Sequenziamento:** Alterazione dolosa dell'ordine di arrivo dei pacchetti. Il protocollo TCP gestisce il riordino a livello di rete, ma se le applicazioni software di livello superiore non implementano controlli sicuri, possono andare in crash o eseguire comandi sfalsati.

Interruzione del Servizio e Scansione delle Porte

L'interruzione compromette la *disponibilità* della rete. A differenza di altre minacce, può manifestarsi sia come blocco totale sia come degradazione parziale delle prestazioni.

Può essere causata da guasti hardware, da malware o da una domanda eccessiva di risorse (DoS). Un ruolo critico è ricoperto dal routing: errori di configurazione o attacchi ai router possono isolare intere reti.

Scansione delle porte (Port Scanning)

È un'attività di **ricognizione passiva**. Non danneggia il sistema, ma prepara il terreno per l'attacco vero e proprio.

- **Come funziona:** Strumenti come *Nmap* inviano pacchetti a un indirizzo IP per "bussare" a tutte le porte logiche (da 1 a 65535).
- **Cosa scopre:** Analizzando le risposte (o la mancanza di risposte) e i tempi di latenza, l'attaccante mappa la topologia della rete, scopre quali porte sono *aperte/chiusa*, deduce quale Sistema Operativo è in uso (OS fingerprinting) e quali servizi (demoni) sono in ascolto.
- **Il problema del Dual-Use:** Il Port Scanning è un'arma a doppio taglio. È usato in modo illecito dagli hacker per trovare vulnerabilità, ma è anche lo strumento principale degli amministratori di sistema per fare audit e mettere in sicurezza la propria infrastruttura. Essendo facilmente automatizzabile tramite script, è impossibile vietarlo.

Sicurezza delle Reti Wireless (WiFi)

Sicurezza delle Reti Wireless (WiFi)

Il Frame WiFi

L'unità fondamentale di trasmissione nel WiFi è il **Frame**, composto da:

1. **MAC Header:** Contiene le istruzioni di instradamento e controllo.
2. **Payload:** I dati effettivi trasmessi.
3. **FCS (Frame Check Sequence):** Un valore usato per il controllo di integrità e rilevamento degli errori di trasmissione.

Elementi chiave del MAC Header:

- **Tipo di frame:** Definisce lo scopo (Control, Management, Data).

- **ToDS / FromDS:** Bit che indicano la direzione del traffico rispetto al *Distribution System* (es. da o verso l'Access Point).
- **Bit di controllo:** Gestiscono la frammentazione e l'ordinamento.
- **Bit WEP:** Indica se il payload è crittografato.
- **Indirizzi MAC:** Può contenere fino a 4 indirizzi (Mittente originario, Destinatario finale, Trasmettitore radio attuale, Ricevitore radio attuale).

Management Frame (Frame di Gestione)

Sono cruciali per instaurare e gestire la connessione. Si dividono in tre tipi principali:

1. **Beacon:** Inviati periodicamente dall'Access Point (AP) in *broadcast* per annunciare la propria esistenza, il proprio SSID e i parametri di sicurezza supportati.
2. **Authentication:** Richieste del client (la scheda di rete) verso l'AP per identificarsi e chiedere il permesso di accesso.
3. **Association:** Segue l'autenticazione. Stabilisce la sessione logica tra client e AP e concorda i parametri crittografici da usare.

SSID (Service Set Identifier): È il "nome" della rete (fino a 32 caratteri). Inserito nei Beacon, permette ai client di distinguere una rete dall'altra in un ambiente affollato. (Spesso include o deriva dal seriale dell'AP per evitare omonimie).

Accesso WiFi Non Autorizzato e Punti Deboli

La procedura di connessione ha falle strutturali:

1. **Intercettazione Beacon:** Sono trasmessi in chiaro. Chiunque può leggerli e clonare l'identità dell'AP.
2. **Autenticazione in Open Mode:** In reti non protette (o reti pubbliche), l'AP accetta chiunque senza verifiche reali. L'associazione con AP malevoli o mal configurati è un rischio costante.

Vulnerabilità Architetture:

- **Open Mode vs Closed Mode:**
 - *Open (SSID Visibile):* L'AP urla il suo nome ai quattro venti.
 - *Closed (SSID Nascosto):* L'AP tace. È il client che deve "urlare" la sua richiesta di ricerca per quello specifico SSID. L'attaccante in ascolto cattura la richiesta del client e simula di essere l'AP cercato (*Evil Twin*).
- **SSID in Chiaro:** Anche in reti chiuse, l'SSID è visibile nell'header dei pacchetti dati una volta stabilita la connessione. Basta sniffare un frame per scoprirlo.
- **MAC Filtering Inutile:** Bloccare gli accessi basandosi su una lista di MAC address consentiti (whitelist) è inefficace. Un attaccante può sniffare un MAC legittimo connesso alla rete e falsificare la propria scheda di rete per usare quello stesso indirizzo (*MAC Spoofing*), aggirando il blocco del router.

L'Evoluzione della Crittografia WiFi

Il Fallimento del WEP (Wired Equivalent Privacy)

Il WEP fu il primo tentativo di rendere il WiFi sicuro quanto un cavo, usando una chiave condivisa.

- **Come autenticava:** L'AP inviava un numero casuale (Challenge) in chiaro. Il client lo cifrava con la chiave WEP e lo rimandava indietro. L'AP verificava se il risultato corrispondeva.

Perché il WEP è stato bucato (Debolezze fatali):

1. **Chiave debole e statica:** Venduto come 64 o 128 bit, in realtà la chiave segreta era solo di 40 o 104 bit, perché 24 bit erano occupati dall'IV (Initialization Vector). Inoltre, la password restava sempre la stessa e spesso era facile da indovinare.
2. **Algoritmo RC4 mal implementato:** Il WEP univa la chiave statica all'IV (trasmesso in chiaro!) per creare il *keystream* usato per cifrare i dati (tramite XOR). Se l'attaccante indovinava anche solo una piccola parte del contenuto del pacchetto originale (es. un header IP noto), poteva invertire l'operazione, calcolare il keystream e decifrare tutto il traffico futuro.
3. **Collisioni dell'IV:** 24 bit permettono solo circa 16 milioni di combinazioni. Su reti trafficate, i valori dell'IV si esaurivano e si ripetevano in pochi minuti. Vedere lo stesso IV due volte permetteva di forzare la crittografia.
4. **Integrità debole (CRC32):** Il controllo di integrità usava l'algoritmo CRC32 (lineare e non crittografico). L'attaccante poteva modificare i dati cifrati e ricalcolare un CRC32 matematicamente valido senza conoscere la chiave.
5. **Autenticazione falsificabile:** Il WEP si fidava del MAC e dell'SSID, entrambi facilmente falsificabili (spoofing).

Il Protocollo WPA (WiFi Protected Access)

Nato per tappare le falle del WEP senza richiedere la sostituzione dell'hardware dei router, ha introdotto 5 miglioramenti vitali:

1. **TKIP (Temporal Key Integrity Protocol):** Addio chiavi statiche. Le chiavi di cifratura cambiano dinamicamente pacchetto per pacchetto.
2. **Autenticazione Robusta (EAP):** Permette l'uso di server aziendali centralizzati (RADIUS) in *WPA-Enterprise* (usando certificati o token), o di password complesse in *WPA-Personal* (PSK - Pre-Shared Key).
3. **Cifratura Forte:** Inizialmente con TKIP, poi evoluta allo standard AES (molto più sicuro) in WPA2 e WPA3.
4. **Integrità Protetta (MIC):** Sostituisce il debole CRC32 con un controllo di integrità crittografico a 64 bit (Message Integrity Code), impedendo la manomissione del testo cifrato.
5. **4-Way Handshake:** Un processo di negoziazione a quattro passaggi matematicamente sicuro per stabilire la sessione ed evitare attacchi replay o intercettazioni della chiave principale durante l'associazione.

Attacchi al WPA:

- **Man in the Middle (Spoofing di Disassociazione):** Nelle vecchie versioni (prima del WPA3), i frame di gestione (come il comando "scollegati") non erano cifrati né firmati. Un attaccante, clonando il

MAC dell'AP, inviava un pacchetto di disconnessione al client. Mentre il client cercava di ricollegarsi, l'hacker rubava il suo MAC e si inseriva nella sessione con l'AP.

- **Autenticazione Incompleta (Evil Twin):** Nelle versioni WPA-Personal, il client dimostra all'AP di conoscere la password, ma non viceversa (manca il "sospetto reciproco"). L'attaccante crea un finto AP con lo stesso nome e segnale più forte; il client vi si connette ciecamente, consegnando l'hash della password.
- **Ricerca Esaustiva (Bruteforce/Dizionario):** Poiché l'algoritmo del WPA2 è solido, l'unico modo per craccarlo (se si usa WPA-PSK) è intercettare l'handshake iniziale e provare milioni di password comuni (dizionario) offline finché non si trova quella che genera lo stesso hash.
- **Attacco ChopChop (solo su TKIP legacy):** Sfrutta vulnerabilità nel controllo di integrità delle prime versioni. Intercetta un pacchetto, ne modifica un byte alla volta e osserva se l'AP lo scarta o meno, deducendo pezzo per pezzo la chiave senza decifrare nulla.

Denial of Service (DoS)

L'obiettivo di un attacco DoS è distruggere la **Disponibilità** di un sistema (rallentandolo o causandone il crash).

Cause/Vettori Generici:

1. **Attacchi volumetrici:** Saturano fisicamente la larghezza di banda (bandwidth) della connessione di rete.
2. **Attacchi applicativi:** Mirano a esaurire risorse logiche (CPU, RAM, thread) del server o dell'app.
3. **Interruzione del collegamento:** Taglio fisico del cavo o disabilitazione della porta di rete.
4. **Guasti HW/SW:** Sfruttamento di bug noti per mandare in panico macchine o programmi.

La Minaccia Principale: Il Flooding (Inondazione)

L'attaccante sommerge il bersaglio con un numero di pacchetti o richieste talmente elevato da paralizzarlo. Spesso si usano pacchetti malformati o tecniche di amplificazione per aggirare i controlli.

- **Ping of Death:** Invio di un pacchetto ICMP (ping) malformato che supera deliberatamente la dimensione massima consentita dal protocollo (65.535 byte). Quando il SO ricevente tenta di riassembalarlo, il buffer va in overflow causandone il crash.
- **Smurf Attack (Amplificazione):** L'attaccante invia una richiesta di ping ICMP (Echo Request) all'indirizzo **broadcast** di una grossa rete terza (la rete complice), ma **falsifica (spoofa)** l'IP mittente mettendo quello della vittima. Tutte le centinaia di macchine della rete complice risponderanno contemporaneamente, ma la risposta non andrà all'attaccante, bensì sommergerà la vittima.
- **Echo-Chargen (Loop Infinito):** Sfrutta servizi diagnostici vecchi. L'attaccante manda un pacchetto dall'Host A (dal servizio Chargen, che genera caratteri a raffica) verso l'Host B (al servizio Echo, che rispedisce indietro tutto ciò che riceve). L'attaccante maschera gli IP affinché A e B pensino che la richiesta provenga l'uno dall'altro. Si innesca una partita di ping-pong infinita che consuma tutta la CPU.
- **SYN Spoofing (Esaurimento Memoria/Connessioni):** Sfrutta il 3-way handshake del TCP.
 1. L'hacker manda milioni di richieste di inizio connessione (pacchetti SYN) con IP falsificati.

2. Il server risponde a ciascuna con un SYN-ACK e alloca una porzione di memoria (nella tabella delle connessioni) mettendola in stato *half-open*, in attesa della conferma finale.
3. L'hacker non manda mai la conferma (l'ACK finale). La tabella delle connessioni del server si riempie in pochi secondi, impedendo l'accesso agli utenti legittimi.

- **Frammentazione IP (Attacco Teardrop):** Sfrutta la funzione del protocollo IP che divide i pacchetti grossi indicando "lunghezza" e "posizione (offset)". L'hacker invia frammenti intenzionalmente sballati, dove gli offset si sovrappongono. Il SO del bersaglio non sa come incastrarli, va in panico e restituisce una schermata blu o un blocco critico (esaurimento risorse di calcolo).

Altri Vettori DoS Complessi

Attacchi al DNS (Risoluzione Nomi) Il DNS converte nomi (es. google.com) in IP. Manipolarlo causa enormi disservizi:

- **DNS Spoofing (MitM):** L'attaccante intercetta la richiesta DNS di un utente e, essendo più veloce del server legittimo, gli risponde fornendo un indirizzo IP falso, dirottandolo su un sito trappola.
- **DNS Poisoning (Avvelenamento Cache):** Mira al Server DNS stesso, non all'utente. L'hacker indovina l'ID sequenziale della richiesta che il server DNS sta facendo verso l'esterno, e inietta una risposta contraffatta. Il server salva questa risposta malevola nella sua Cache. Da quel momento, chiunque chieda a quel server l'indirizzo del sito, verrà dirottato automaticamente.
- **Attacchi TLD (Top Level Domain):** Colpiscono l'infrastruttura mondiale dei root server DNS per far crollare Internet. (Oggi quasi impossibile grazie al routing *Anycast* che distribuisce il carico globalmente).

Dirottamento di Sessione (TCP Hijacking) A differenza del MitM classico, qui l'hacker non si mette "in mezzo", ma *subentra*. Dopo che l'utente ha fatto login e stabilito una sessione TCP, l'hacker intercetta i *Sequence Number* e gli *Acknowledgment Number* scambiati. Calcolando la sequenza corretta, l'hacker inietta pacchetti al server usando il MAC/IP della vittima. Il server crede di parlare ancora con l'utente legittimo, mentre l'utente originale viene desincronizzato e disconnesso ("tagliato fuori").

Attacchi al Routing Manipolano le "mappe stradali" (le tabelle di instradamento) dei router.

- Se un router compromesso mente, dicendo agli altri router vicini: "Posso raggiungere qualsiasi destinazione in un solo salto", tutto il traffico globale convergerà su di lui, saturandolo (*Blackholing* o DoS totale) o permettendogli intercettazioni massicce.
- **Source Routing:** Una funzione IP (solitamente disattivata oggi) che permette al mittente di dettare l'esatto percorso fisico del pacchetto. Se usata malevolmente, forza i dati a transitare attraverso router compromessi.

Reflection Attack nelle Reti (DDoS)

Nel contesto delle reti, un attacco di riflessione è una tecnica usata per inondare una vittima di traffico (Denial of Service) nascondendo l'identità dell'attaccante e, spesso, amplificando il danno.

Come funziona: L'attaccante non invia il traffico malevolo direttamente alla vittima, ma usa dei server legittimi di terze parti come "specchi" (i *reflector*).

1. L'attaccante invia una richiesta a un server legittimo (es. un server DNS o NTP).

2. Nel farlo, **falsifica l'indirizzo IP mittente (IP Spoofing)**, inserendo l'indirizzo IP della vittima al posto del proprio.
3. Il server legittimo elabora la richiesta e invia la risposta alla vittima, credendo che sia stata lei a chiederla.

DDoS e Botnet

Un DDoS è un DoS lanciato contemporaneamente da decine di migliaia di PC.

- **La Botnet:** Una rete di computer, server o dispositivi IoT ignari ("Zombie/Bot"), precedentemente infettati tramite malware (spesso Trojan silenti).
- **Gerarchia C&C:** I bot non attaccano a caso, obbediscono al **Botmaster** (l'hacker). Gli ordini passano attraverso server di Comando e Controllo (C&C), strutturati in modo ridondante per non essere smantellati dalle autorità.
- **Modalità di comando:**
 - **Push:** Il server C&C contatta i bot e invia l'ordine d'attacco.
 - **Pull:** I bot interrogano silenziosamente il C&C o domini generati dinamicamente (DGA - **Domain Generation Algorithm**) per scaricare le istruzioni.
- **Contromisure (DoS/DDoS):**
- **Patching:** Aggiornare costantemente i server per chiudere le falle usate per la diffusione dei malware che creano botnet.
- **Protezione Attiva:** Creazione teorica di software/agenti mobili difensivi (Autonomous Mobile Protective Agents) che simulano il comportamento dei bot per infiltrarsi nella botnet e disattivare i trojan infetti dall'interno.
- **Gestione del Traffico:**
 - **Tuning (Ottimizzazione):** Abbassare i timeout TCP (per liberare in fretta le risorse allocate per SYN flood) o usare i **SYN Cookies** (che non allocano memoria finché l'handshake non è completo).
 - **Load Balancing:** Distribuire il traffico d'attacco su reti immense a livello globale (come Cloudflare o Akamai) tramite **Anycast** o **CDN**, assorbendo l'onda d'urto in modo che i singoli server non crollino.
 - **Shunning / Blacklisting:** Rilevare l'attacco e bloccare completamente a livello di router il traffico proveniente dalle subnet ostili o applicare regole di rate-limiting severo.

Crittografia di Rete e Modello OSI

La protezione dei dati cambia radicalmente a seconda del livello della pila OSI in cui viene applicata la crittografia.

1. Link Encryption (Cifratura di Linea - Livello 1/2 OSI):

- Cifra il pacchetto grezzo subito prima di immetterlo nel cavo fisico o nell'aria.
- È totale, invisibile e trasparente sia all'utente che ai livelli software superiori.

- **Punto debole:** Il pacchetto deve essere fisicamente decifrato e riportato in chiaro all'interno di *ogni singolo router* intermedio per poterne leggere l'header IP e capire dove instradarlo. Se un router intermedio è compromesso, i dati sono esposti.

2. End-to-End Encryption (Cifratura End-to-End - Livello 4/7 OSI):

- Cifra solo il contenuto (payload) del pacchetto.
- I dati partono cifrati dal mittente e vengono decifrati *solo* quando arrivano al computer del destinatario finale.
- Gli header logici (IP e TCP) restano in chiaro durante tutto il tragitto, permettendo ai router intermedi di instradarli senza doverli aprire. Protegge contro i router compromessi, ma espone informazioni sul mittente/destinatario.

3. Applicazioni e Browser (Livello 7/4 OSI):

- **SSH (Secure Shell):** Standard industriale per l'amministrazione remota. Sostituisce il vecchio Telnet in chiaro. Instaura un tunnel crittografato e autenticato per inviare comandi testuali al SO remoto. Prevede una fase iniziale di negoziazione per accordarsi sulle chiavi (es. via Diffie-Hellman) e sui metodi di login.
- **SSL/TLS (HTTPS):** Lavora al livello di Trasporto (L4). Protegge il traffico web (e altri). Esegue un *handshake* per negoziare algoritmi e scambiare chiavi simmetriche per la sessione. Si basa profondamente sulla **PKI (Public Key Infrastructure)**: utilizza *Certificati Digitali* firmati da *CA (Certificate Authority)* fidate per garantire al client che il server a cui si sta connettendo sia legittimo e non un attaccante MitM.

4. IPsec (Livello 3 - Rete):

- Nato per IPv6, supportato in IPv4. È lo standard primario per la creazione di VPN sicure.
- Rende inefficaci attacchi di spoofing IP, intercettazioni e dirottamenti di sessione crittografando o firmando interi pacchetti IP.
- **SA (Security Association):** È il cuore di IPsec. È un accordo *unidirezionale* (A verso B) che stabilisce policy, chiavi e protocolli. (Quindi, per una comunicazione bidirezionale servono due SA). Ogni SA è tracciata tramite un numero identificativo univoco chiamato **SPI (Security Parameter Index)**.

Firewall

Architettura di Rete e Segmentazione

La segmentazione della rete riduce drasticamente l'esposizione degli host all'esterno, isolando i sistemi sensibili in sottoreti protette. Queste comunicano con l'esterno unicamente tramite punti di controllo, come un gateway *dual-homed* (con due schede di rete: una interna e una esterna).

Tuttavia, il semplice routing controlla i flussi macroscopici ma non realizza un *controllo degli accessi granulare* sui contenuti o sugli stati delle connessioni; per questo obiettivo serve un Firewall.

Il Firewall

Un Firewall è un dispositivo hardware e/o software posizionato a livello perimetrale che filtra il traffico tra una rete interna (fidata/protetta) e una rete esterna (non fidata, es. Internet), implementando specifiche e rigorose politiche di sicurezza.

Per ridurre la superficie d'attacco, il SO del firewall (spesso un'appliance dedicata) è appositamente "castrato": privato di interfacce grafiche inutili, compilatori e strumenti di programmazione sfruttabili da un hacker in caso di breccia.

L'amministratore deve configurare il dispositivo basandosi su due logiche contrapposte (comportamenti predefiniti):

- **Default Permit:** Tutto ciò che non è esplicitamente vietato, è permesso (più permissivo e rischioso).
- **Default Deny:** Tutto ciò che non è esplicitamente permesso, è vietato (più sicuro, è lo standard di fatto).

Progettazione del Firewall

L'efficacia di un firewall è dovuta a un design strutturale affidabile e a una politica di controllo dei flussi rigorosamente definita.

- **Politica di Sicurezza:** Prevede un insieme di regole logiche valutate dall'alto verso il basso (top-down, la prima regola che fa "match" viene applicata). Non esiste uno standard universale: ogni amministratore configura le proprie regole in base alle esigenze aziendali.
- **Il concetto di Fiducia (Reference Monitor):** Il firewall è un esempio perfetto di applicazione del concetto di *Reference Monitor*:
 1. *Sempre invocato:* È posizionato fisicamente o logicamente in un punto di sbarramento obbligatorio; tutto il traffico deve attraversarlo.
 2. *Immune da manomissioni:* È isolato su un computer/appliance dedicato e fortificato.
 3. *Piccolo e Semplice (relativamente):* Svolge una funzione mirata, garantendo la massima analizzabilità e affidabilità del suo codice.

Tipi di Firewall (In base al Livello OSI e logica)

1. Packet Filtering (Filtro a pacchetti - L3 Rete):

- *Logica:* **Stateless** (Senza memoria).
- Prende ogni pacchetto di dati e ne legge solo l'etichetta esterna (l'Header), che contiene l'Indirizzo IP (sorgente/destinazione), la Porta e il Protocollo (TCP, UDP). Se questi dati coincidono con le sue regole, lo lascia passare.
- Veloce ed efficiente, ma "cieco" al contesto. Non controlla il payload e non ricorda lo stato della connessione (non sa se un pacchetto ACK è legittimo o fa parte di un attacco spoofing).

2. Stateful Inspection (Ispezione di Stato - L3/L4):

- *Logica:* Ricorda il passato.
- Fa tutto ciò che fa il packet filter, ma tiene una tabella in memoria dello *stato* di ogni connessione (es. "L'host interno ha appena mandato un SYN, quindi mi aspetto di vedere un SYN-ACK in risposta da quello specifico IP").

- Previene attacchi complessi (SYN flood frammentati, UDP spoofing) perché blocca i pacchetti fuori sequenza o non correlati a connessioni legittime tracciate.

3. Circuit-Level Gateway (L5 Sessione):

- Interviene *solo all'inizio* della comunicazione. Controlla minuziosamente l'Handshake e l'autenticazione per assicurarsi che l'utente sia chi dice di essere.
- Una volta che la connessione è stata giudicata sicura e legittima, il firewall smette di controllare. Crea un "tubo" diretto tra te e il server e lascia fluire i dati alla massima velocità, fidandosi ciecamente.

4. Application-Proxy Gateway (L7 Applicazione):

- È il sistema standard più sicuro e paranoico, ma per questo motivo anche il più lento. Lavora al livello più alto, capendo perfettamente il linguaggio del programma che stai usando (es. capisce se stai scaricando un file via FTP o navigando su un sito via HTTP).
- Interrompe fisicamente la connessione. L'utente interno si collega al proxy (È un intermediario totale. Con questo firewall, tu non sei **mai** connesso direttamente al server su Internet). Il proxy decodifica il protocollo (es. comprende esattamente cosa sia una richiesta HTTP GET o FTP PUT) e, se autorizzata, instaura una *nuova e distinta connessione* col server esterno per inoltrare la richiesta.
- Ispeziona il payload e previene l'esecuzione di comandi applicativi anomali.

5. Deep Packet Inspection (DPI) & Next-Gen Firewall:

- Apice dell'analisi. Guarda "dentro" il pacchetto per analizzare il payload effettivo alla ricerca di pattern di malware noti, applicativi non autorizzati (es. blocca BitTorrent indipendentemente dalla porta usata) e fa traffic shaping avanzato (prioritizzazione di banda).

6. Guard:

- Più potente di un Proxy L7. Non si limita a scartare un pacchetto malevolo, ma può *modificarne* il contenuto al volo prima di consegnarlo (es. sanificare un URL da una potenziale SQL Injection). L'estrema complessità del suo software lo rende, ironicamente, vulnerabile a sua volta.

7. Network Sandboxing:

- Un ambiente di detonazione virtuale controllato, spesso integrato ai Next-Gen Firewall. Se il firewall riceve un file sospetto (es. un eseguibile mai visto prima), lo devia nella Sandbox, lo esegue e osserva cosa fa il programma (Zero-day analysis comportamentale). Se è innocuo, lo lascia passare.

Limitazioni Strutturali dei Firewall:

- Se un utente aggira fisicamente il perimetro (es. attivando l'hotspot del telefono connesso al PC aziendale), il firewall perimetrale diventa inutile (Bypass).
- Non proteggono i dati mentre viaggiano su Internet dopo essere usciti dall'azienda.
- Essendo i "guardiani del cancello", sono costantemente esposti al fuoco nemico (bersaglio ad alta visibilità).
- Efficacia legata al 100% all'errore umano: una regola mal scritta rende l'hardware inutile.
- Zero controlli interni: se l'infezione (es. chiavetta USB) parte da dentro, un firewall perimetrale base non la blocca.

NAT (Network Address Translation) e Topologie

- **NAT:** Maschera tutta l'infrastruttura di rete. I dispositivi interni hanno IP privati (es. 192.168.x.x). Quando un pacchetto esce verso Internet, il firewall ne sostituisce l'IP sorgente con l'unico (o pochi) IP Pubblico assegnato all'azienda, memorizzando l'associazione in una tabella. Nasconde la topologia della rete, rendendo impossibili gli attacchi diretti mirati agli IP dei singoli PC interni.
- **Same on Separate LAN (Configurazione Base):** Il firewall non viene mai collegato direttamente al cavo Internet. Sta in mezzo tra due router: un *Border Router* (esterno, che filtra lo sporco macroscopico) e un Router Interno (che dirige il traffico pulito nella LAN privata).
- **La DMZ (Demilitarized Zone):** È una terza area di rete, isolata sia da Internet che dalla LAN aziendale sicura. Contiene i server che *devono* essere esposti al pubblico (es. Web Server, Mail Server aziendale, Application Proxy). Se un hacker compromette il Web Server nella DMZ, non ha un accesso diretto ai server database critici nascosti nella LAN aziendale retrostante.

IDS (Intrusion Detection System)

L'IDS è un analizzatore **passivo**. Osserva il traffico o i log di sistema, scova anomalie e *avvisa l'amministratore (Allarme)*. Non prende provvedimenti per bloccare l'attacco in corso.

Funzionalità base dell'IDS:

- Verifica integrità file vitali.
- Riconoscimento di attacchi noti (tramite firme).
- Rilevamento di deviazioni statistiche (anomalie comportamentali).
- Gestione dei log e integrazione con Honeypot per analizzare gli intrusi.

Tipologie di Rilevamento (Come lavorano)

1. Signature-Based (Basati su Firma):

- Cercano stringhe esatte o pattern conosciuti di attacchi documentati nei pacchetti.
- *Pro:* Rilevamento perfetto per minacce note (es. vecchi exploit). Nessun falso positivo su traffico normale.
- *Contro:* Assolutamente inutili contro attacchi mai visti prima (*Zero-day*) o contro malware leggermente offuscati/polimorfi che sfalsano la firma. Faticano a ricostruire attacchi frammentati come il Teardrop.

2. Heuristic / Anomaly-Based (Euristico/Comportamentale):

- Monitorano il sistema basandosi sull'idea che un'infezione avvenga per accumulo di piccoli eventi "strani". Un *Inference Engine (Motore Inferenziale)* fa scattare l'allarme quando la somma di anomalie supera una soglia di tolleranza.
- **State-based:** Controlla che le configurazioni critiche del SO o i file non cambino stato in modo anomalo.
- **Model-based:** Crea un profilo di comportamento "normale" per la rete (baseline) tramite Machine Learning o storici di rete. Se un utente inizia improvvisamente a scaricare giga di dati

alle 3 del mattino, lo rileva come deviazione dal modello.

- **Pro:** Identifica malware sconosciuti (zero-day) e minacce interne.
- **Contro:** Tende a generare troppi **Falsi Positivi** (traffico legittimo bloccato per un picco anomalo di utilizzo), richiedendo molto tuning umano per stabilire la baseline iniziale.

L'analisi avanzata (Stateful Protocol Analysis): Crea una vera e propria macchina a stati logica per ricostruire l'intero contesto della comunicazione, tracciando attacchi lenti e distribuiti nel tempo. È la tecnica più accurata, ma richiede enormi risorse di RAM e CPU.

Posizionamento dell'IDS (Dove lavorano)

- **HIDS (Host-Based):** Software installato sul singolo PC/Server. Controlla i file log locali, l'integrità del registro e le chiamate di sistema (System Calls). Ideale per capire *cosa succede dentro la macchina*, ma se l'hacker diviene Root/Admin, può semplicemente spegnere il software HIDS.
- **NIDS (Network-Based):** Apparato hardware dedicato. Analizza il traffico di intere reti esaminando il payload dei pacchetti. È invisibile agli hacker (non ha un IP pubblico a cui connettersi). Raccoglie telemetria da firewall e sensori sparsi.

Deploy Fisico del NIDS:

- **Frontend IDS:** Posizionato fuori dalla rete interna, tra internet e il firewall. Vede *tutti* gli attacchi in tempo reale, ma è altamente esposto (può subire attacchi DoS lui stesso).
- **Internal IDS:** Posizionato dentro la LAN. Vede solo ciò che il firewall non ha filtrato, indicando minacce interne reali o falle del firewall stesso. Totalmente protetto e invisibile dall'esterno.

Stealth Mode e Valutazione: Per proteggere l'appliance IDS, la si configura in **Stealth Mode**: si usano due schede fisiche di rete. La prima è in "modalità promiscua", priva di indirizzo IP, e collegata alla rete dati: serve solo come orecchio per ascoltare il traffico, senza poter parlare o essere contattata. La seconda scheda, usata per mandare gli allarmi all'amministratore, è collegata a una rete logica separata, sicura e inaccessibile (Out-of-Band).

- Il successo dell'IDS si misura bilanciando **Falsi Positivi** (Allarmi finti = stancano l'analista) e **Falsi Negativi** (Attacchi non rilevati = rete bucata). È difficile trovare il giusto punto di calibrazione della sensibilità, e l'ultima parola spetta sempre a un essere umano.

Intrusion Prevention System (IPS)

È l'evoluzione attiva dell'IDS. Non si limita ad urlare "Al lupo!", ma **spara**. L'IPS è posizionato *in-line* (il traffico ci passa obbligatoriamente attraverso) e implementa risposte automatiche in tempo reale alle minacce rilevate, senza aspettare l'intervento umano.

Risposte Autonome dell'IPS: Le Policy di reazione dell'IPS si dividono in quattro comportamenti:

1. **Monitoraggio:** Se l'affidabilità dell'allarme è bassa, l'IPS non blocca (per evitare di buttare fuori utenti legittimi) ma intensifica l'analisi e la raccolta dati su quell'IP.
2. **Protezione (Blocco Attivo):** Intervento drastico. Tronca le connessioni TCP inviando pacchetti di Reset (RST), inibisce temporaneamente l'accesso a directory specifiche o revoca la sessione utente in corso.

3. **Segnalazione Dinamica:** Comunica istantaneamente con gli altri apparati di rete (es. manda un comando API al firewall dicendogli: "Aggiungi subito questo IP siberiano in blacklist"). Innalza il DEFCON generale della rete.
4. **Chiamata a un operatore:** Genera un ticket prioritario per il Security Operations Center (SOC), scalando la problematica all'intervento umano per anomalie complesse e indecifrabili.